

INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN:

AN AGILE, ITERATIVE APPROACH

SATZINGER | JACKSON | BURD

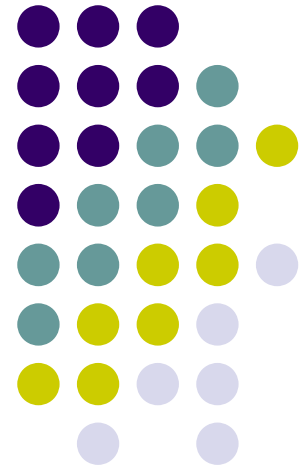
Chapter 11

Object-Oriented Design: Use Case Realizations

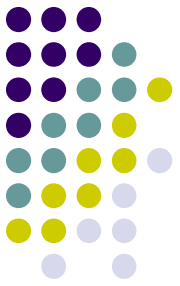
Chapter 11

Introduction to Systems
Analysis and Design:
An Agile, Iterative Approach
6th Ed

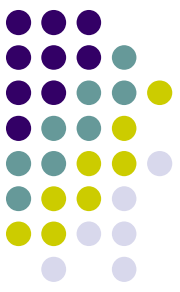
Satzinger, Jackson & Burd



Chapter 11 Object-Oriented Design: Use Case Realizations



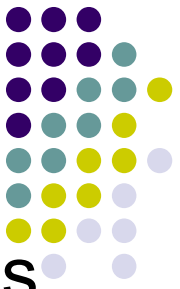
- Detailed Design of Multilayer Systems (多层系统的细节设计)
- Use Case Realization with **Sequence Diagrams** (用例实现和顺序图)
- Designing with **Communication Diagrams** (用协作图进行设计)
- Updating and Packaging the **Design Classes** (更新和打包设计类图)
- Design Patterns (设计模式)



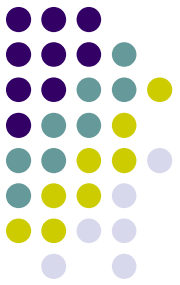
Learning Objectives

- Explain the different types of objects and layers in a design （解释设计中不同种类的目标和层次）
- Develop sequence diagrams for use case realization （为实现用例开发顺序图）
- Develop communication diagrams for detailed design （为细节设计开发协作图）
- Develop updated design class diagrams （开发更新的设计类图）
- Develop multilayer subsystem packages （开发多层子系统包）

Learning Objectives



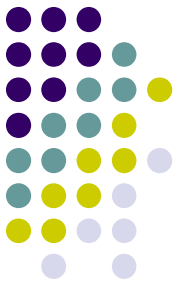
- Explain design patterns and recognize various specific patterns (解释设计模式并识别不同的特殊模式)
- Chapter 10 introduced software design concepts for OO programs, multi-layer design, use case realization using the CRC cards technique, and fundamental design principles (10 章介绍了面向对象程序的软件设计概念、多层设计、CRC 卡技术、基本设计原则)
- This chapter continues the discussion of OO software design at a more advanced level (本章继续在更高级的层面上讨论面向对象软件设计)



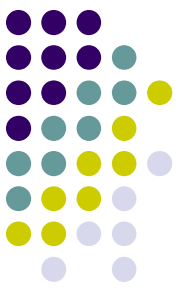
Chapter 11 Outline

- Detailed Design of Multilayer Systems (多层系统的细节设计)
- Use Case Realization with Sequence Diagrams
- Designing with Communication Diagrams
- Updating and Packaging the Design Classes
- Design Patterns

Detailed Design of Multilayer Systems

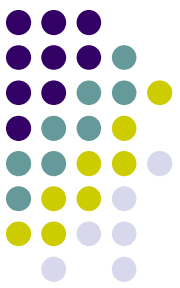


- CRC Cards focuses on the business logic, also known as problem domain layer of classes (CRC 卡主要关注业务逻辑, 也就是类的问题域层)
- Three layers include **view layer, business logic/problem domain layer, and data access layer** (本章关注的三层系统包括视图层、业务逻辑 / 问题域层和数据访问层)



Design Patterns

- Design Pattern (设计模式) — standard design techniques and templates that are widely recognized as good practice (被广泛认为是良好实践的标准设计技术和模板)
- For common design/coding problems, the design pattern suggests the best way to handle the problem.
(对于常见的设计 / 编码问题, 设计模式建议了处理问题的最佳方法)



Design Patterns

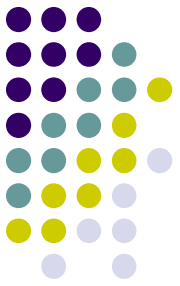
- 程序设计模式分为创建型模式、结构型模式、行为型模式三种：
 - **创建型模式**：包括工厂方法模式、抽象工厂模式、单例模式、建造者模式、原型模式
 - **工厂方法模式**：定义一个用于创建对象的接口，让子类决定实例化哪一个类， Factory Method 使得一个类的实例化延迟到了子类
 - **抽象工厂模式**：提供一个创建一系列相关或相互依赖对象的接口，而无需制定他们的具体类
 - **单例模式**：保证一个类只有一个实例，并提供一个访问它的全局访问点
 - **建造者模式**：将一个复杂对象的构建与它的表示相分离，是的同样的构建过程可以创建不同的表示
 - **原型模式**：用原型实例制定创建对象的种类，并且通过拷贝这些原型来创建新的对象

Factory Design Pattern

(工厂模式)

Name:	Factory or Factory Method
Problem:	Who should be responsible for creating utility type objects that do not specifically belong to the problem domain classes? These utility objects may also be accessed from various places within the system, so a given object may need to be instantiated from several classes.
Solution:	Create an artifact that is a factory class. Its responsibility is only to instantiate utility classes. In many situations, only one instance of a particular utility class is allowed. Hence, all classes that need access to the class come through the factory. The factory ensures that only one instance is created.
Example:	Several places in the RMO system need to get data from an Order object and need to have a reference to an Order_DA [data access] object. The Order_DA object may or may not already have been instantiated. A data access factory is defined and an interface is created. The requesting object uses the methods defined in the interface to request the reference to the Order_DA object. It then can read the database of orders. <pre> public synchronized Order_DA getOrder_DA () { if (myODA == null) { myODA = new Order_DA (); } return myODA; } </pre>
Benefits and Consequences:	Higher cohesion of problem domain classes Less coupling between business logic layer and data layer Smaller, more maintainable classes

Factory Design Pattern (工厂模式)



问题：谁应该为创建了不明确属于问题域类的工具类型对象负责？

（在设计中经常需要工具类，包括数据访问对象或者控制器类。什么类需要创建这些工具对象呢？多数情况下，域类创建他们没有意义，因为这不是域类的责任之一）

解决方案：创建一个工厂类。它的责任仅仅是实例化工具类

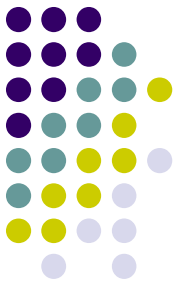
样例：RMO 系统的几个位置需要从订单对象中活动数据并获得 Order_DA（数据访问）对象的引用。Order_DA 对象不知是否已经实例化。定义一个数据访问工厂并创建一个界面。请求的对象是用在界面中被定义的方法以获取 Order_DA 对象的引用。然后它就能读取订单的数据库

优点和结果：问题域类具有更高的内聚性是业务逻辑和数据层之间的耦合更少
更小、更加可维持的类

Singleton Design Pattern (单例)

Name:	Singleton
Problem:	Only one instantiation of a class is allowed. The instantiation (new) can be called from several places in the system. The first reference should make a new instance, and later attempts should return a reference to the already instantiated object. How do you define a class so that only one instance is ever created?
Solution:	A singleton class has a static variable that refers to the one instance of itself. All constructors to the class are private and are accessed through a method or methods, such as getInstance(). The getInstance() method checks the variable; if it is null, the constructor is called. If it is not null, then only the reference to the object is returned.
Example:	In RMO's system, the connection to the database is made through a class called Connection. However, for efficiency, we want each desktop system to open and connect to the database only once, and to do so as late as possible. Only one instance of Connection—that is, only one connection to the database—is desired. The Connection class is coded as a singleton. The following coding example is similar to C# and Java: <pre> Class Connection { private static Connection conn = null; public synchronized static getConnection () { if (conn == null) { conn = new Connection (); } return conn; } } </pre> Another example of a singleton pattern is a utilities class that provides services for the system, such as a factory pattern. Because the services are for the entire system, it causes confusion if multiple classes provide the same services. An additional example might be a class that plays audio clips. Since only one audio clip should be played at one time, the audio clip manager will control that. However, for this to work, there must be only one instance of the audio clip manager.
Benefits and consequences:	There are other times when only one instance of an object is needed, but if it is instantiated from only one place, then a singleton may not be required. The singleton object controls itself and ensures that only one instance is created—no matter how many times it is called and wherever the call occurs in the system. The code to implement the singleton is very simple, which is one of the desirable characteristics of a good design pattern.

Singleton Design Pattern (单例)



问题：一个类只允许是用一个实例。这个实例可能被系统的许多地方调用。（例如，一个工厂类或者主窗口类，管理系统资源的服务类，例如数据库连接）

解决方案：单例类有一个静态变量指向它自己的一种实例。所有类的构造器都是私有的，可以通过一种或多种方法防伪，比如 `getInstance`（）

样例：在 RMO 系统重，数据库的连接是通过一个名为 `connection` 的类。然而，就效率而言，我们想要每个桌面系统与数据库仅连接一次，并尽可能快。只有一种情况下可以连接，那就是被请求时。
连接类被编译成一个单例

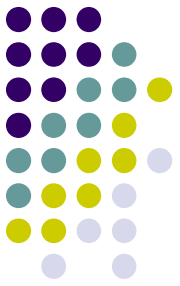
优点：在某些时间只需要一个对象实例，但如果它只从一个地方实例化，那么我们可能不需要这个单例。这个单例对象控制它自己并确保值创建了一个实例，无论它被调用多少次，无论它在系统的什么地方出现

Singleton Design Pattern (单例)

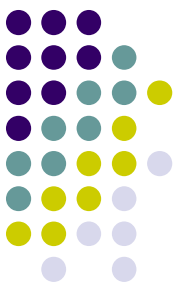
synchronized 是 Java 中的关键字，是利用锁的机制来实现同步的。

锁机制有如下两种特性：

- **互斥性**：即在同一时间只允许一个线程持有某个对象锁，通过这种特性来实现多线程中的协调机制，这样在同一时间只有一个线程对需同步的代码块（复合操作）进行访问。互斥性我们也往往称为操作的原子性。
- **可见性**：必须确保在锁被释放之前，对共享变量所做的修改，对于随后获得该锁的另一个线程是可见的（即在获得锁时应获得最新共享变量的值），否则另一个线程可能是在本地缓存的某个副本上继续操作从而引起不一致。



Design Patterns

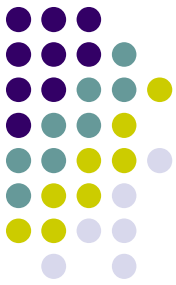


结构型模式：适配器模式、装饰器模式、代理模式、外观模式、桥接模式、组合模式、享元模式。

- (1) 适配器模式：将一类的接口转换成客户希望的另外一个接口，适配器使得原本由于接口不兼容而不能一起工作的那些类可以一起工作。
- (2) 装饰器模式：动态地给一个对象增加一些额外的职责，就增加的功能来说，装饰器模式相比生成子类更加灵活。
- (3) 代理模式：为其他对象提供一种代理以控制对这个对象的访问。
- (4) 外观模式：为子系统中一组接口提供一致的界面，此模式定义了一个高层接口，该接口使得子系统更加容易使用。
- (5) 桥接模式：将抽象部分与它的实现部分相分离，使它们可以独立地变化。
- (6) 组合模式：将对象组合成树形结构以表示部分整体的关系，组合使得用户对单个对象和组合对象的使用更具有一致性。
- (7) 享元模式：运用共享技术有效地支持大量细粒度的对象。

Adapter Design Pattern

适配器



Name:	Adapter
Problem:	A class must be replaced, or is subject to being replaced, by another standard or purchased class. The replacing class already has a predefined set of method signatures that are different from the method signatures of the original class. How do you link in the new class with a minimum of impact so that you don't have to change the names throughout the system to the method names in the new class?
Solution:	Write a new class, the adapter class, which serves as a link between the original system and the class to be replaced. This class has method signatures that are the same as those of the original class (and the same as those expected by the system). Each method then calls the correct desired method in the replacement class with the method signature. In essence, it "adapts" the replacement class so that it looks like the original class.
Example:	There are several places in the RMO system where class libraries were purchased to provide special processing. These purchased libraries provide specialized services such as tax calculations and shipping and postage rates. From time to time, these service libraries are updated with new versions. Sometimes a service library is even replaced with one from an entirely different vendor. The RMO systems staff applies protection from variations and indirection design principles by placing an adapter in front of each replaceable class. <pre> classDiagram class System class TaxCalculatorIF { <<interface>> getSTax () getUTax () } class TaxCalcAdapter { getSTax () getUTax () } class ABCTaxCalculator { findTax1 () findTax2 () } System --> TaxCalculatorIF TaxCalculatorIF < -- TaxCalcAdapter TaxCalcAdapter --> ABCTaxCalculator </pre>
Benefits and consequences:	The adaptee class can be replaced as desired. Changes are confined to the adapter class and do not ripple through the system. Two classes are defined, an interface class and the adapter class. Passed parameters may add more complexity, and it is difficult to limit changes to the adapter class.

Adapter Design Pattern 适配器



问题：一个类必须被另一个标准类替代。替换类已经具有一组预定义的方法签名，这些方法签名不同于原始类的方法签名。如何以最小的影响链接新类，从而不必在整个系统中将名称更改为新类中的方法名称

解决方法：写一个新类（适配器类），这个类作为初始系统和被替换的类之间的连接。这个类的方法特征与初始类相似。每个方法要求替换类有正确的方法特征。

样例：在 RMO 系统的许多地方，需要购买类库来提供特别处理。

这些类库可以提供特殊服务，比如计算税务和运输包裹的费用。

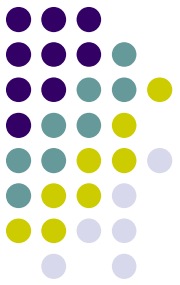
有时候，这些服务库被更新成新的版本。有时候，一个服务库甚至被替换给一个完全不同的供应商。

RMO 系统的员工通过在每个替换部署一个适配器间接链接被替换的类

优点和结果：被改造的类能被替换成需要的类型。适配器类的改变是有限的，并且不会与系统连锁

已通过的参数可能变得更复杂，并且很难限制适配器类的改变

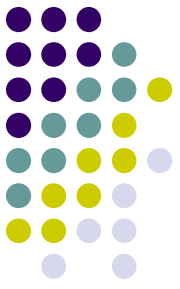
Design Patterns



行为型模式：控制器模式、观察者模式、模板方法、命令模式、状态模式、策略模式、职责链模式、中介者模式、访问者模式、解释器模式、备忘录模式。

- (1) 控制器模式：提供一种方法，顺序访问一个聚合对象的各个元素，而又不需要暴露该对象的内部表示。
- (2) 观察者模式：定义对象间一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知自动更新。
- (3) 模板方法：定义一个操作中的算法的骨架，而将一些步骤延迟到子类中，模板方法使得子类可以不改变算法的结构即可重定义该算法的某些特定步骤。
- (4) 命令模式：将一个请求封装为一个对象，从而可以用不同的请求对客户进行参数化，对请求排队和记录请求日志，以及支持可撤销的操作。

Design Patterns



- (5) 状态模式：允许对象在其内部状态改变时改变他的行为。对象看起来似乎改变了他的类。
- (6) 策略模式：定义一系列的算法，把它们一个个封装起来，并使它们可以互相替换，本模式使得算法可以独立于使用它们的客户。
- (7) 职责链模式：使多个对象都有机会处理请求，从而避免请求的送发者和接收者之间的耦合关系
- (8) 中介者模式：用一个中介对象封装一些列的对象交互。
- (9) 访问者模式：表示一个作用于某对象结构中的各元素的操作，使得可以在不改变各元素类的前提下定义作用于这个元素的新操作。
- (10) 解释器模式：给定一个语言，定义其文法的一个表示，并定义一个解释器，这个解释器使用该表示来解释语言中的句子。
- (11) 备忘录模式：在不破坏对象的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态。

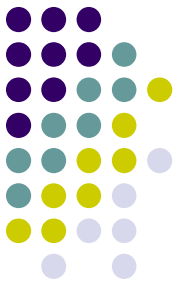
Controller Pattern

控制器模式

First step toward multilayer architecture

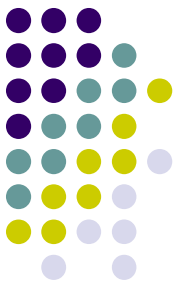
More design patterns are at the end of the chapter

Name:	Controller
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.
Example:	The RMO Customer account subsystem accepts inputs from a <code>:CustomerForm</code> window. These input messages are passed to the <code>:CustomerHandler</code>, which acts as the switchboard to forward the message to the correct problem domain class. <pre> graph LR subgraph UI [User interface] RMO[RMO New Customer] CF[:CustomerForm] RMO -- "createNewCustomer()" --> CF end subgraph DC [Domain classes] CH[:CustomerHandler] C[:Customer] CH -- "createNewCustomer()" --> C end CF -- "createNewCustomer()" --> CH style UI fill:none,stroke:none style DC fill:none,stroke:none </pre> Other cases of the controller pattern will be used for each RMO use case.
Benefits and Consequences:	Coupling between the view layer and the domain layer is reduced. The controller provides a layer of indirection. The controller is closely coupled to many domain classes. If care is not taken, controller classes can become incoherent, with too many unrelated functions. If care is not taken, business logic will be inserted into the controller class.



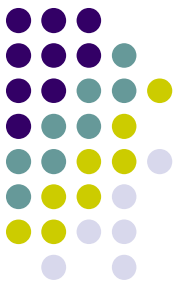
Controller Pattern

Name:	Controller 1. 域类负责处理用例，然而，由于一个用例中可能有许多域类，哪一个负责接收消息？
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.



Controller Pattern

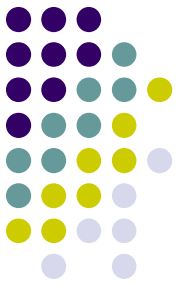
Name:	Controller 2. 用户界面类如果有所有域类的可见性，那么它将显得很复杂。用户界面和域类的耦合度要如何降低？
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.



Controller Pattern

Name:	Controller
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.

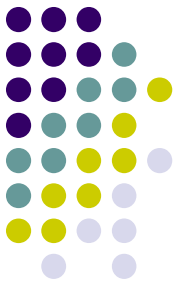
设置一个专门的类接收所有输入消息并充当交换机，
然后转发到正确的域类，有几种解决方式：



Controller Pattern

Name:	Controller
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.

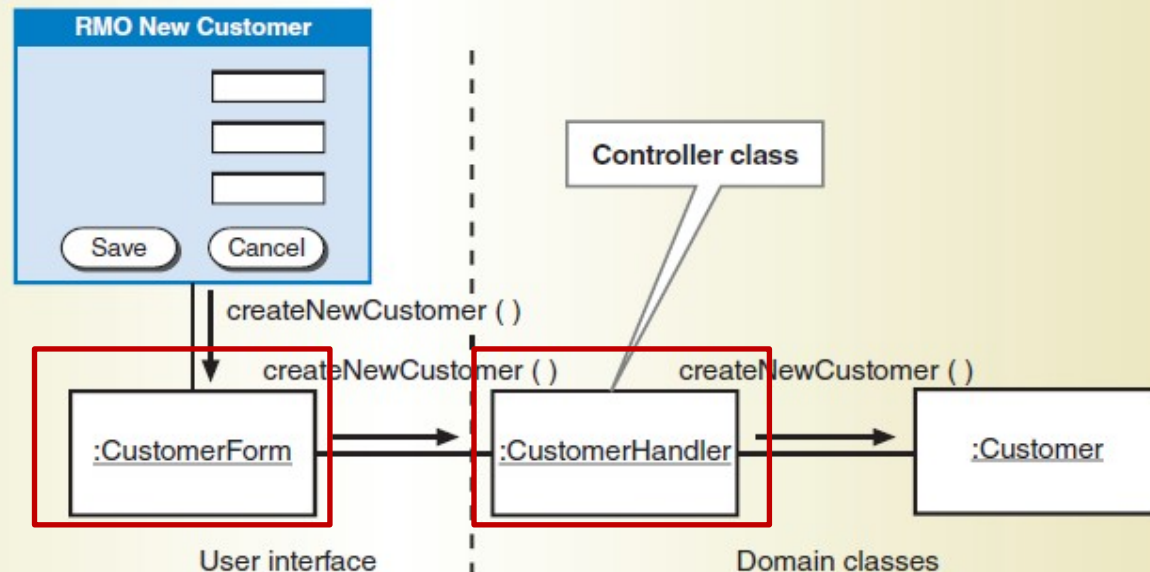
- a) 通过单一的控制器类来表示整个系统
- b) 为每个用例创建一个控制器类



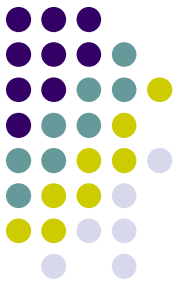
Controller Pattern

Example:

The RMO Customer account subsystem accepts inputs from a `:CustomerForm` window. These input messages are passed to the `:CustomerHandler`, which acts as the switchboard to forward the message to the correct problem domain class.



Other cases of the controller pattern will be used for each RMO use case.



Controller Pattern

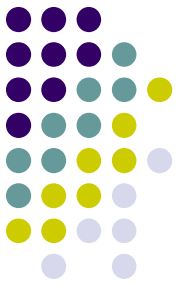
Benefits and Consequences:

Coupling between the view layer and the domain layer is reduced.
The controller provides a layer of indirection.

The controller is closely coupled to many domain classes.
If care is not taken, controller classes can become incoherent,
with too many unrelated functions.
If care is not taken, business logic will be inserted into the controller class.

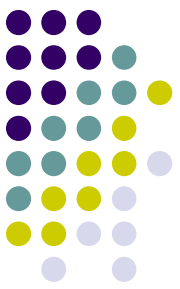
优点：
视图层和域层的耦合度减小了
控制器提供了间接的层

结果：
控制器和许多域类紧密耦合
如果不注意，控制器会变得不合逻辑
如果不注意，业务逻辑将会嵌入控制器中



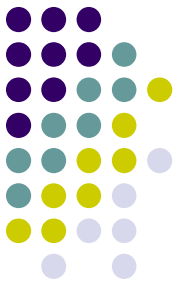
Design Patterns

- Design patterns became widely accepted after the publication of *Elements of Reusable Object-Oriented Software*, (1996) by Gamma et al (the “Gang of Four”) (设计模式在 Elements 这本书发表后被广泛接受)
- There are architectural design patterns talked about already
 - Three layer or model-view-controller architecture (三层架构或模型 - 视图 - 控制器架构)



Design Patterns

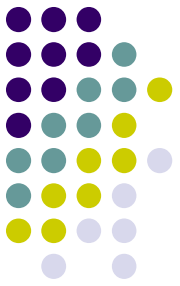
- The first example of a programming design pattern shown is the Controller Pattern (控制器模式) .
 - Problem is deciding how to handle all of the messages from the view layer to classes in the problem domain layer to reduce coupling (问题是决定如何处理从视图层到问题域层中的类的所有消息, 以减少耦合)
 - Solution is to assign one class between the view layer and the problem domain layer that receives all messages and acts as a switchboard directing messages to the problem domain (解决方案是在视图层和问题域层之间分配一个类, 它接收所有消息并充当将消息定向到问题域的交换机)



Chapter 11 Outline

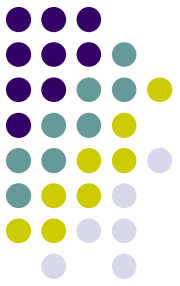
- Detailed Design of Multilayer Systems
- Use Case Realization with Sequence Diagrams (用例实现和顺序图)
- Designing with Communication Diagrams
- Updating and Packaging the Design Classes
- Design Patterns

Use Case Realization with Sequence Diagrams



- Use case realization (用例实现) — the process of elaborating the detailed design of a use case with interaction diagrams (用交互图详细说明用例的详细设计的过程)
- Two types of interaction diagrams
 - UML sequence diagram (emphasized in text) (顺序图)
 - UML communication diagram (also introduced) (通信图)

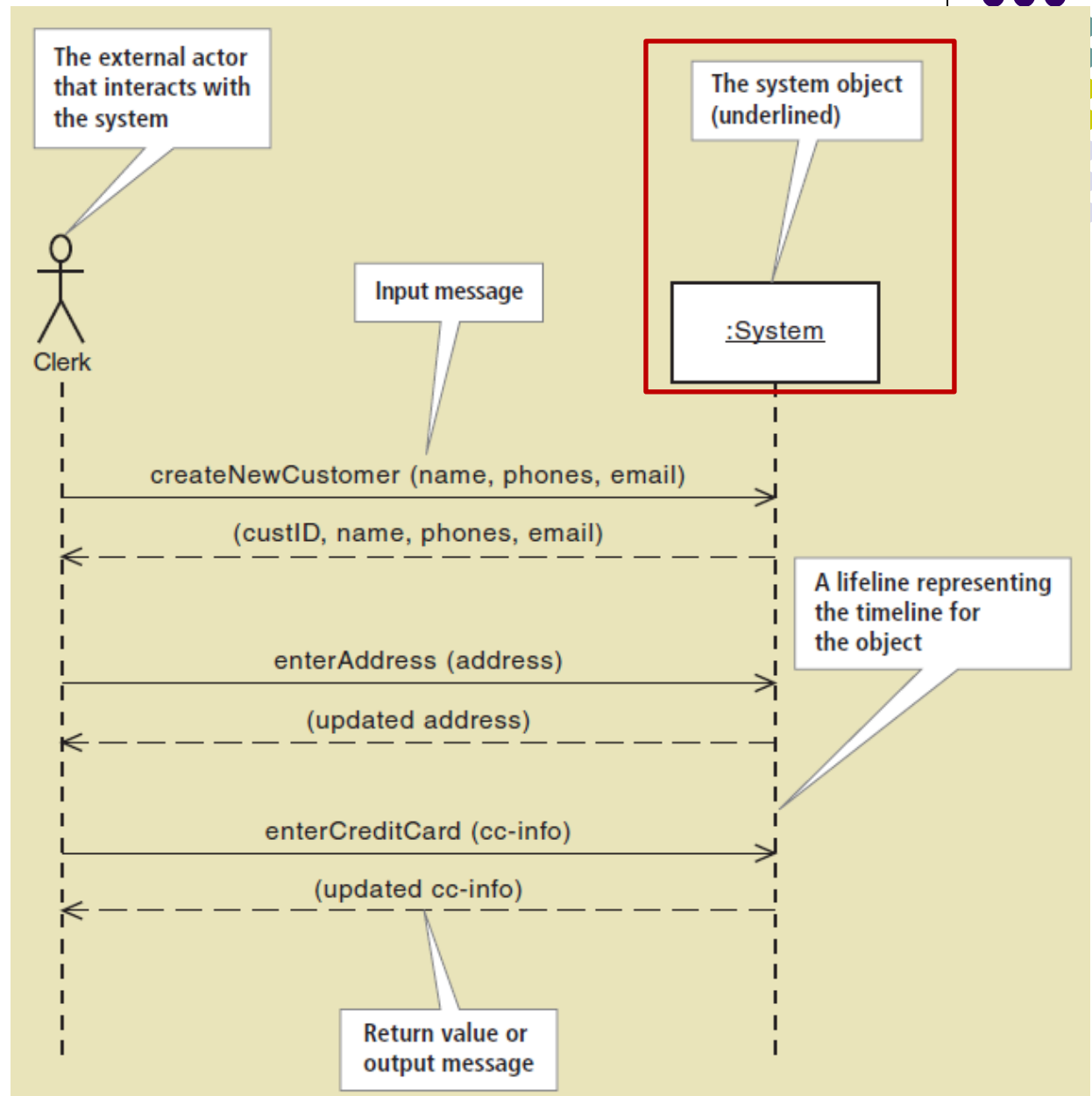
Use Case Realization with Sequence Diagrams



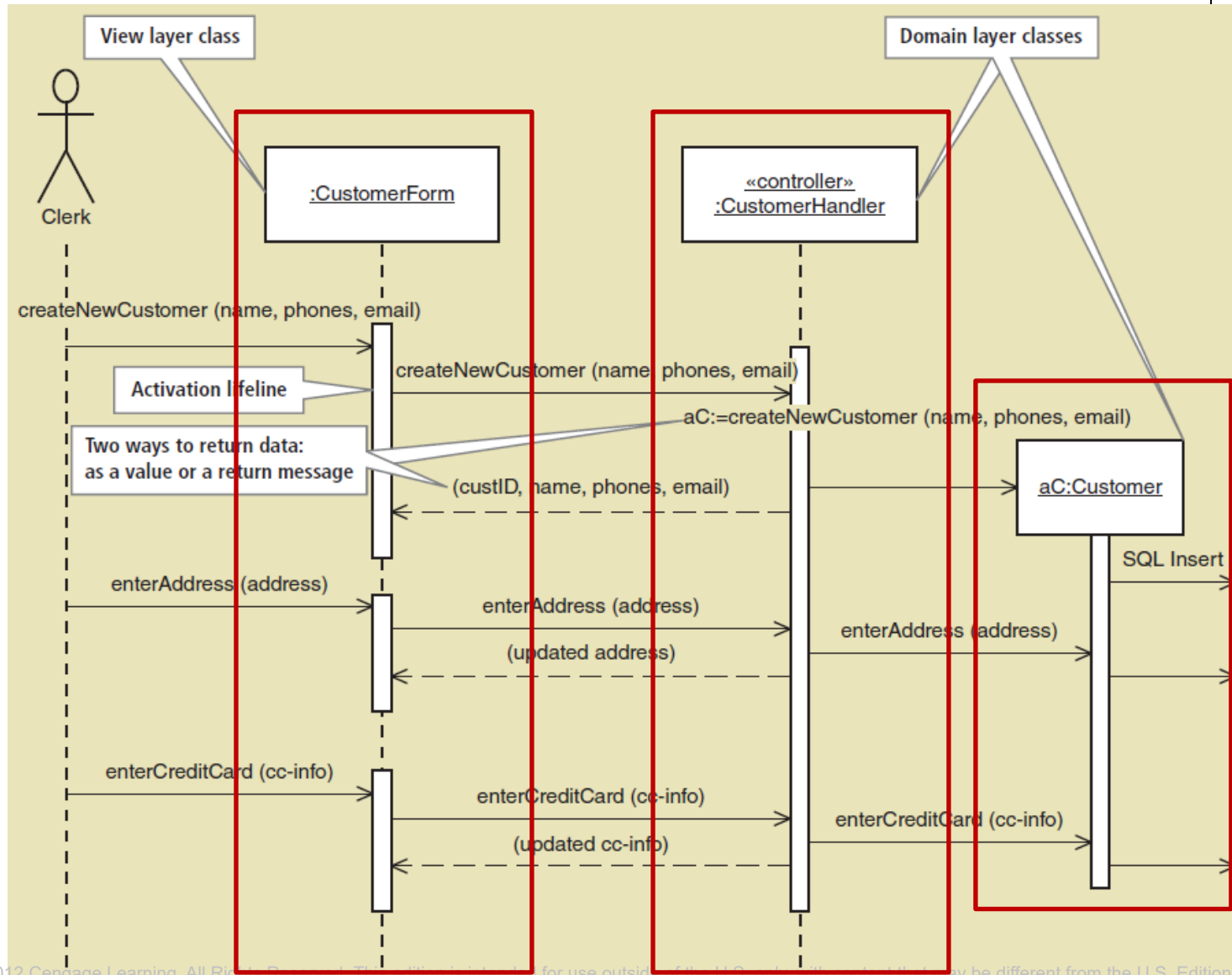
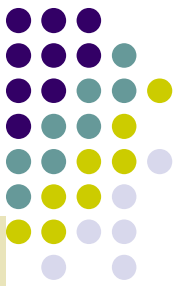
- Sequence diagrams, aka use case realization sequence diagrams, extend the system sequence diagram (SSD) to show:
 - View layer objects
 - Domain layer objects (usually done first)
 - Data access layer objects
- 顺序图，又名用例实现，是通过扩展第四章的系统顺序图 (SSD) 来展示的，包括：
 - 视图层对象
 - 域层对象
 - 数据访问层对象

Start with System Sequence Diagram (SSD)

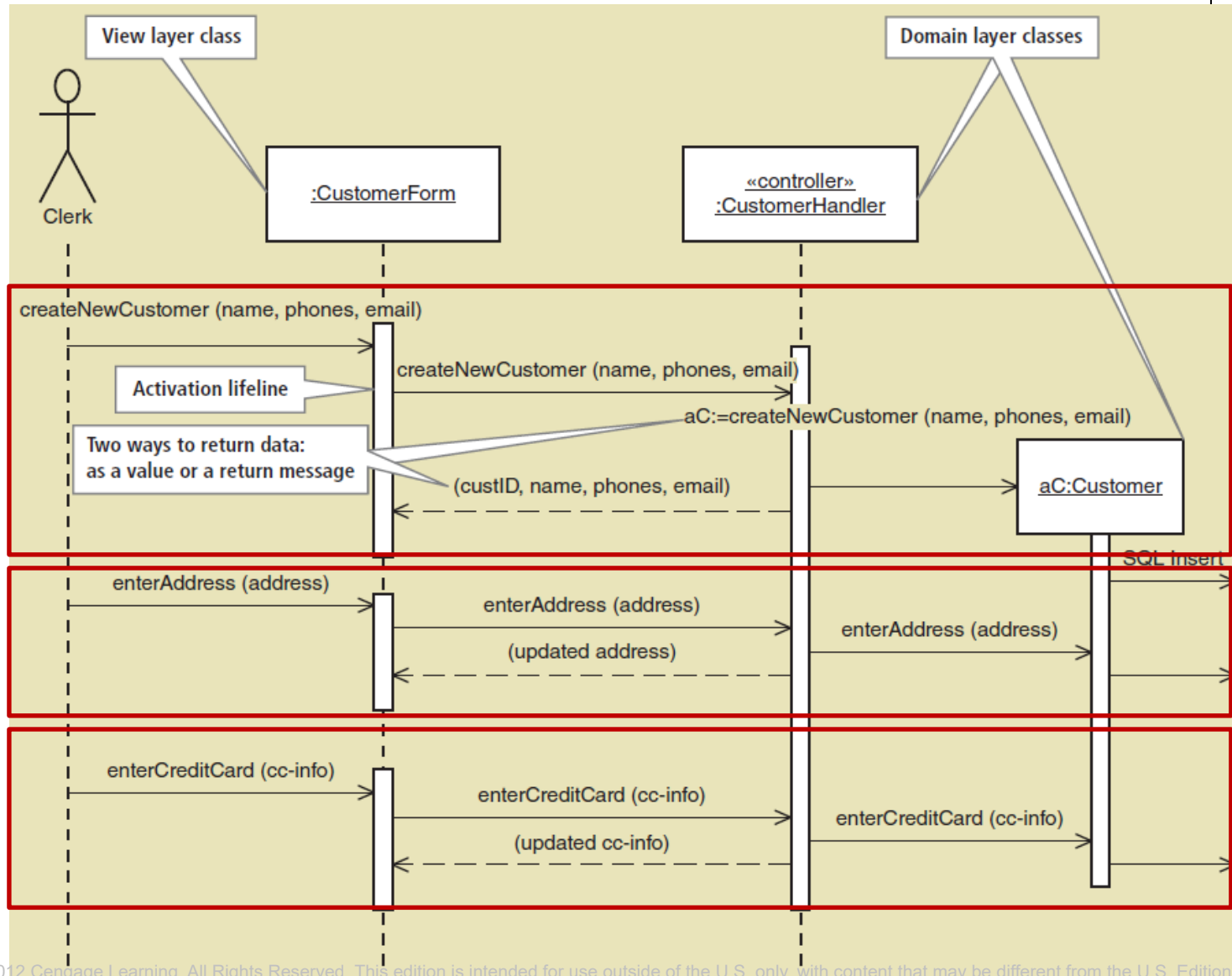
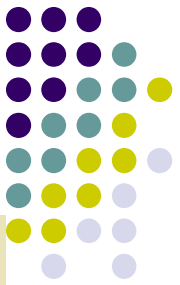
Use case *Create customer account*



Sequence Diagram to show View Layer and Part of Problem Domain Layer

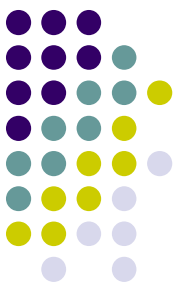


Sequence Diagram to show View Layer and Part of Problem Domain Layer



Notes of Expanded Sequence Diagram

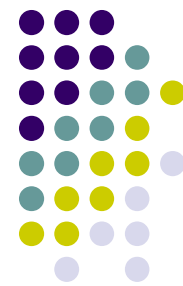
(扩展顺序图注意事项)



- This is a two layer architecture, as the domain class Customer knows about the database and executes SQL statements for data access (这是一个两层体系结构，因为域类 Customer 了解数据库并执行 SQL 语句进行数据访问)
- Three layer design would add a data access class to handle the database resulting in higher cohesiveness and loose coupling (三层设计将添加一个数据访问类来处理数据库，从而产生更高的内聚性和松耦合)

Notes of Expanded Sequence Diagram

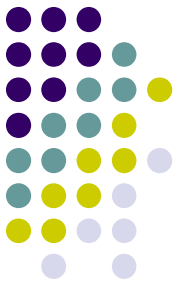
(扩展顺序图注意事项)



- Note :CustomerForm is an object of the CustomerForm class, :CustomerHandler is an object of the CustomerHandler class playing the role of a controller stereotype (both underlined because they are objects) (注意 :CustomerForm 是 CustomerForm 类的一个对象, CustomerHandler 是 CustomerHandler 类的一个对象, 扮演一个控制器原型的角色 (两者都下划线, 因为它们是对象))
- aC:Customer is an object of the Customer class known by reference variable named aC (aC : Customer 是 Customer 类的一个对象, 通过名为 aC 的引用变量来引用)
- Note: the create message to aC:Customer points to the object symbol to indicate create (创建消息给 aC: 客户指向对象符号表示创建)

Overview of Detailed Design Steps

Shown previously in Chapter 10

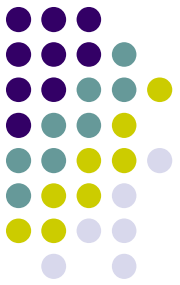


Design Step	Chapter
1. Develop the first-cut design class diagram showing navigation visibility.	10
2. Determine the class responsibilities and class collaborations for each use case using CRC cards.	10
3. Develop detailed sequence diagrams for each use case. (a) Develop the first-cut sequence diagrams. (b) Develop the multilayer sequence diagrams.	11
4. Update the DCD by adding method signatures and navigation information.	11
5. Partition the solution into packages, as appropriate.	11

3. 为每个用例开发详细的顺序图
- a) 开发初步顺序图
 - b) 开发多层顺序图

Overview of Detailed Design Steps

Shown previously in Chapter 10

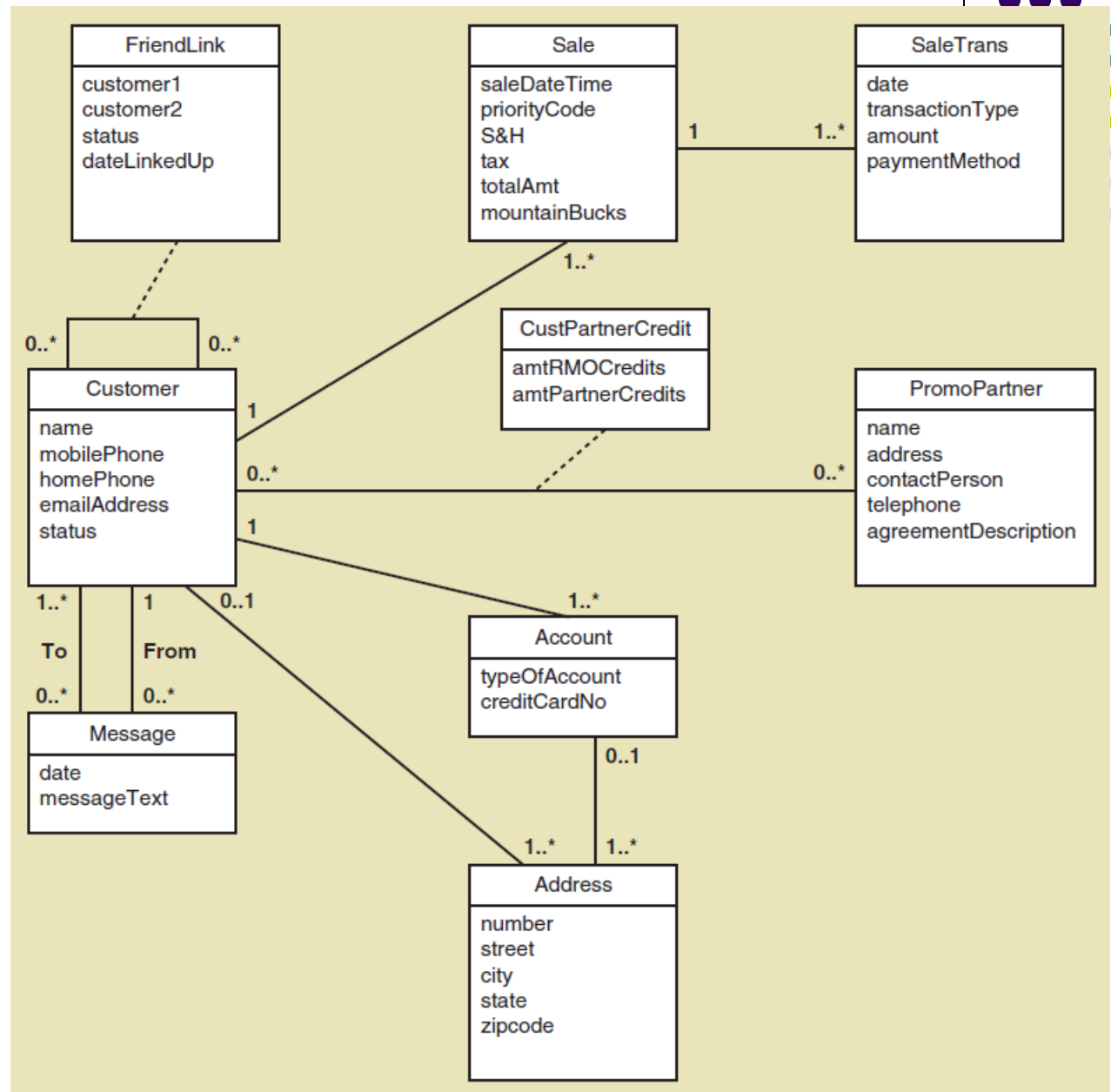


Design Step	Chapter
1. Develop the first-cut design class diagram showing navigation visibility.	10
2. Determine the class responsibilities and class collaborations for each use case using CRC cards.	10
3. Develop detailed sequence diagrams for each use case. (a) Develop the first-cut sequence diagrams. (b) Develop the multilayer sequence diagrams.	11
4. Update the DCD by adding method signatures and navigation information.	11
5. Partition the solution into packages, as appropriate.	11

通过使用 CRC 卡或顺序图，添加方法特征和导航信息，以此来更新设计类图

Create Customer Account Use Case

Start with domain model for **Customer Account Subsystem** (客户账户子系统)

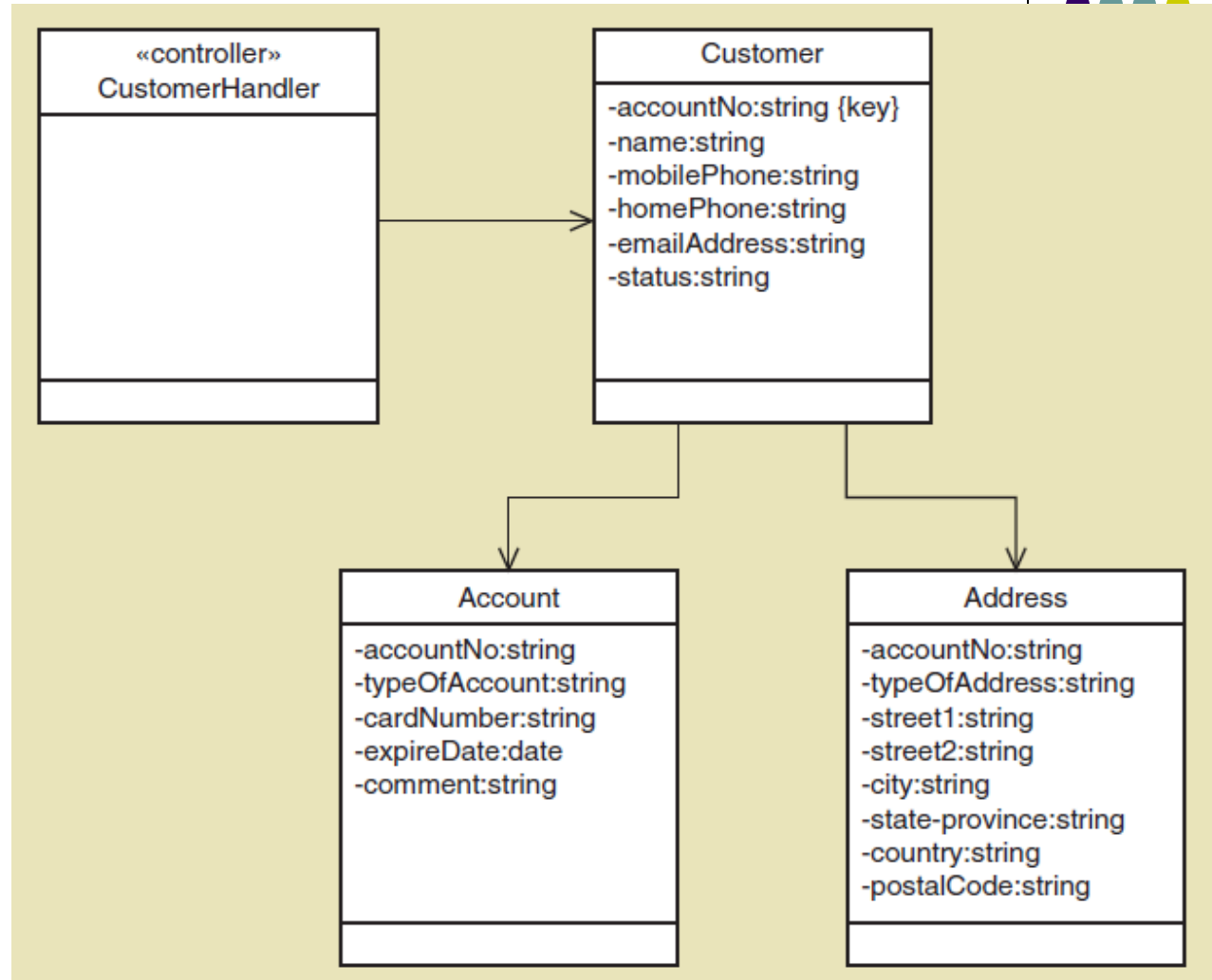




Create Customer Account Use Case

First cut design
class diagram for
use case

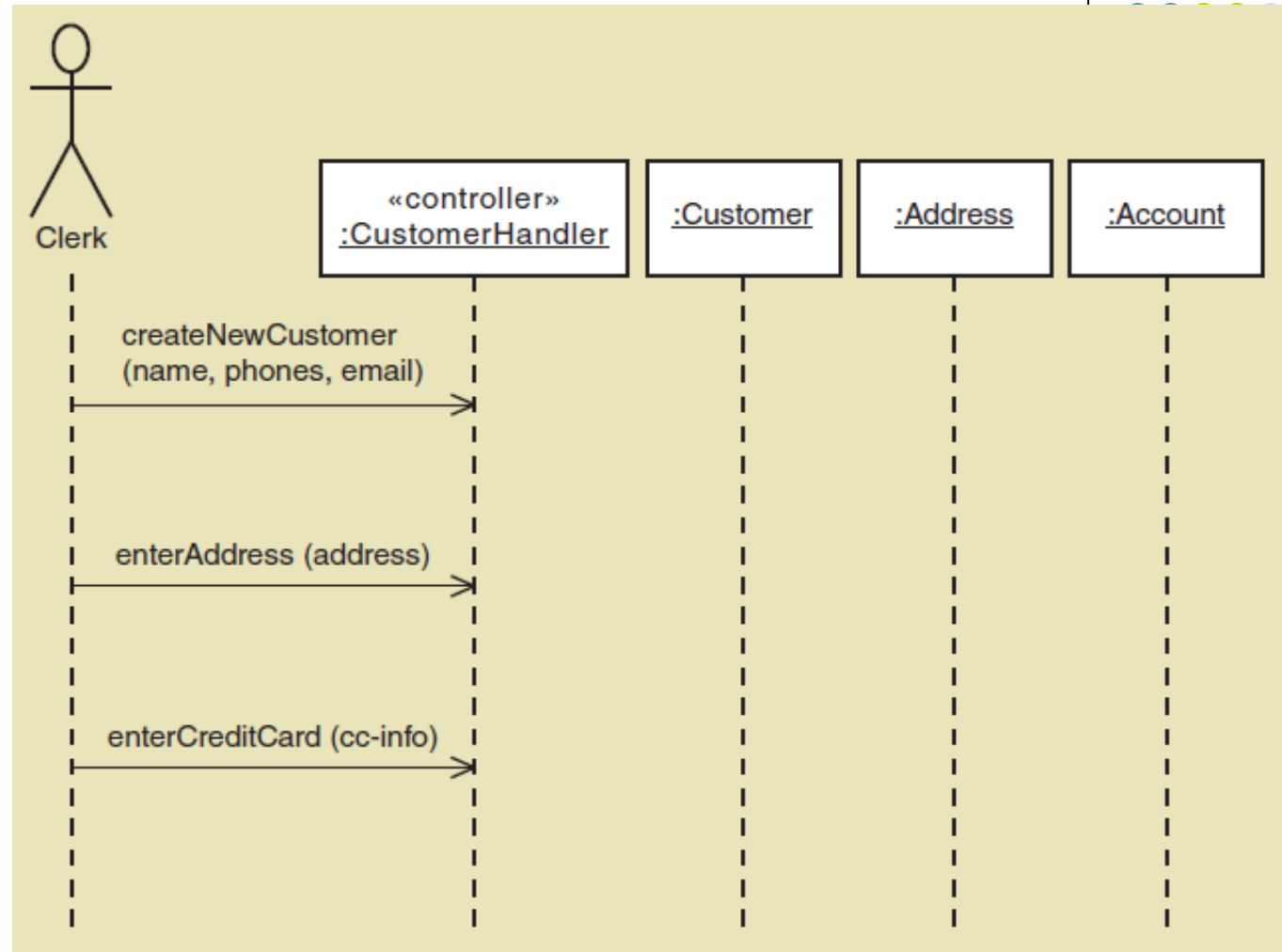
Select needed
classes, elaborate
attributes, add
controller, and add
navigation visibility
选择需要的类、精细的
属性、添加控制器和添
加导航可见性





Create Customer Account Use Case

First cut
sequence
diagram
expanding SSD,
adding controller,
and adding
needed classes

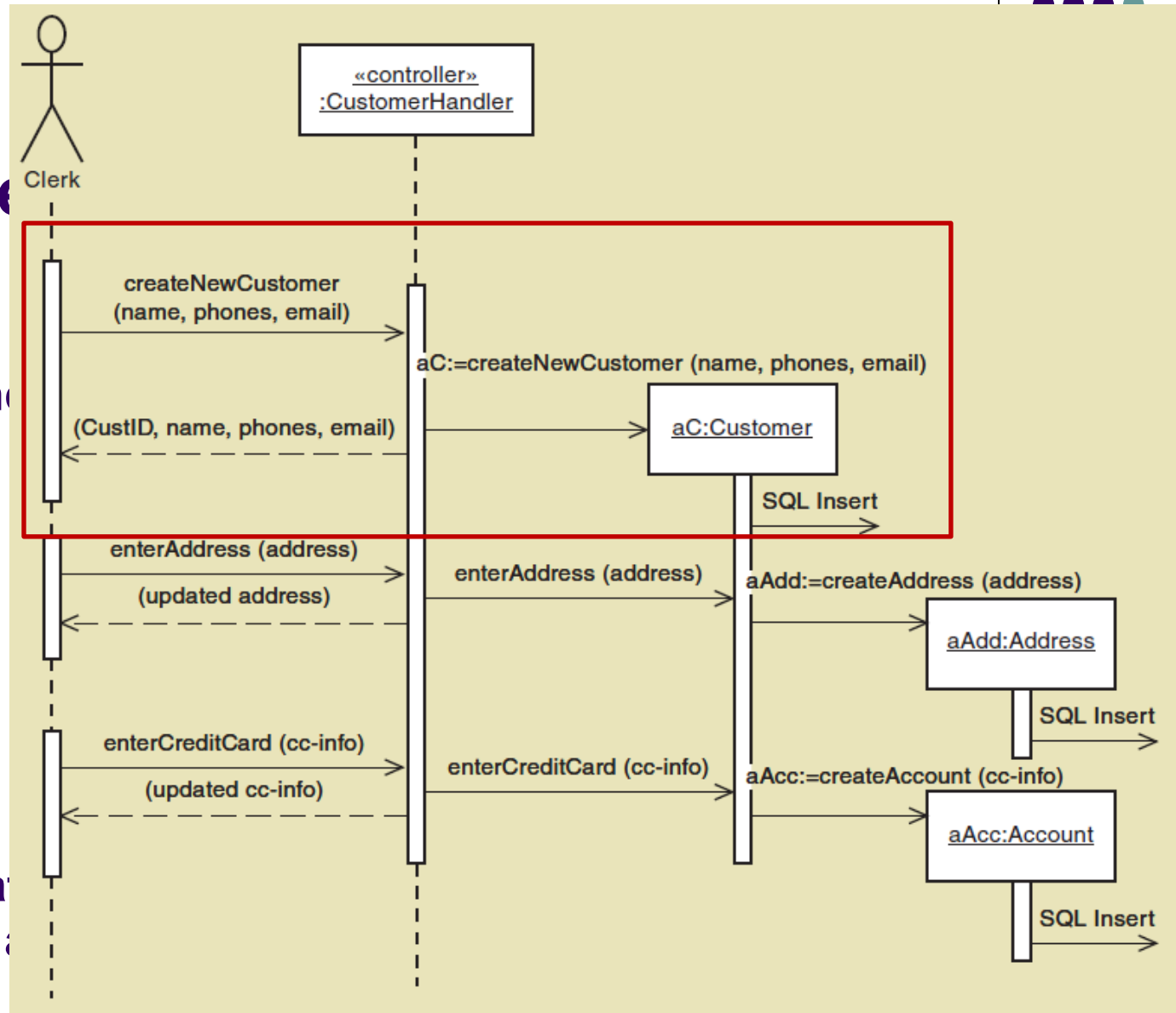


Create Customer Account Use Case

Add messages and activation to complete collaboration

This is just the domain layer

These domain classes handle data access, so this is a two layer architecture

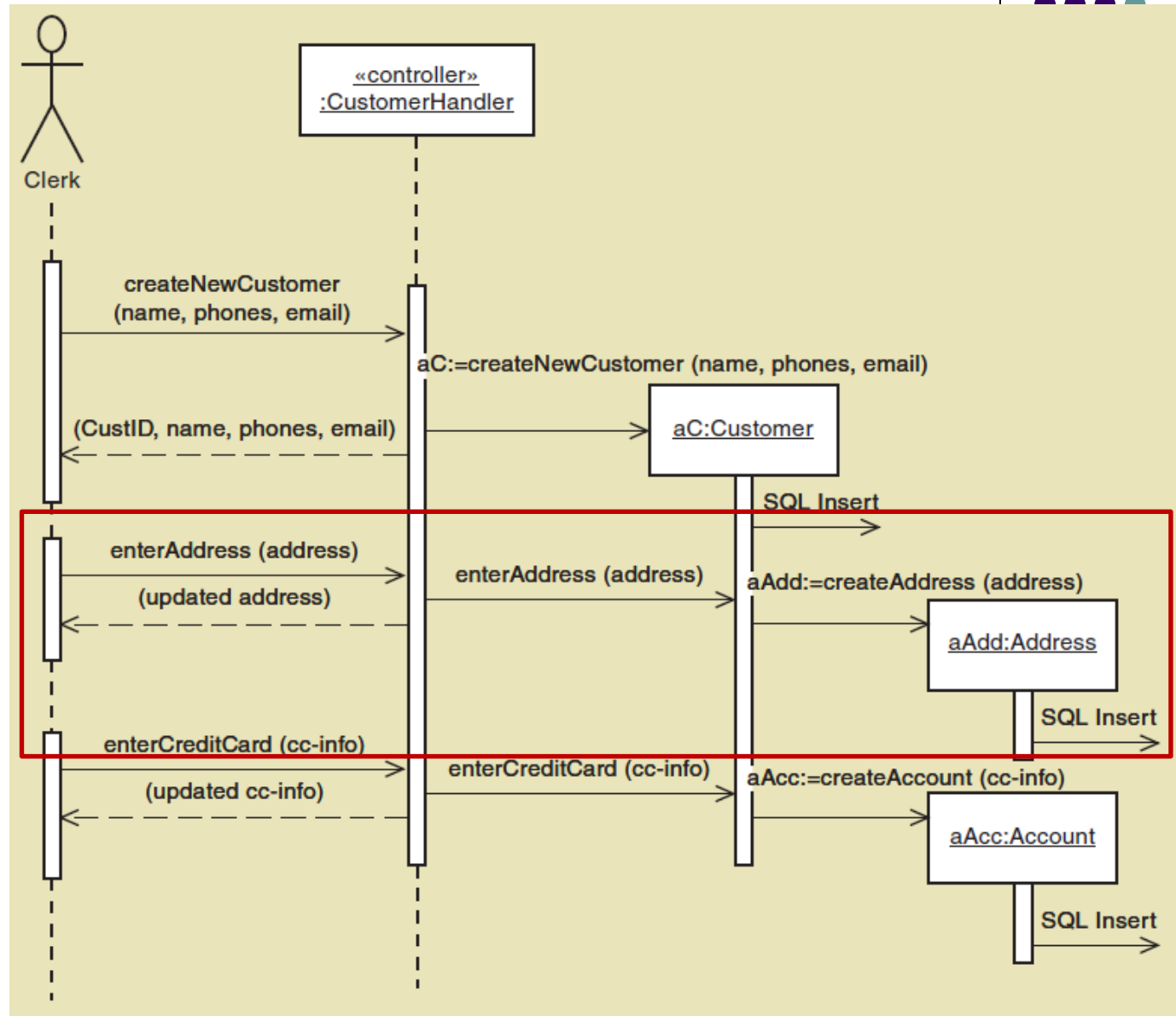


Create Customer Account Use Case

Add messages and activation to complete collaboration

This is just the domain layer

These domain classes handle data access, so this is a two layer architecture

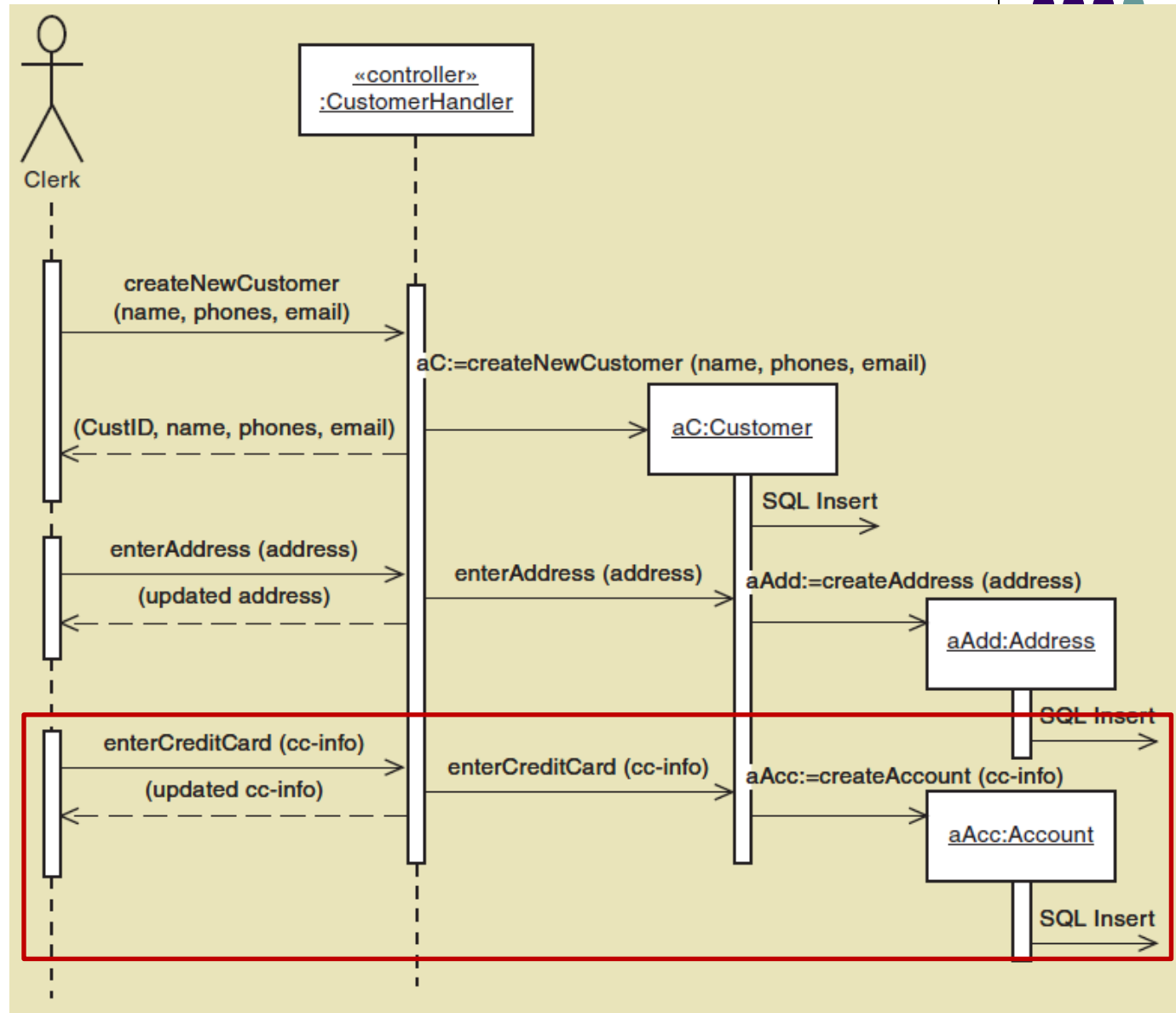


Create Customer Account Use Case

Add messages and activation to complete collaboration

This is just the domain layer

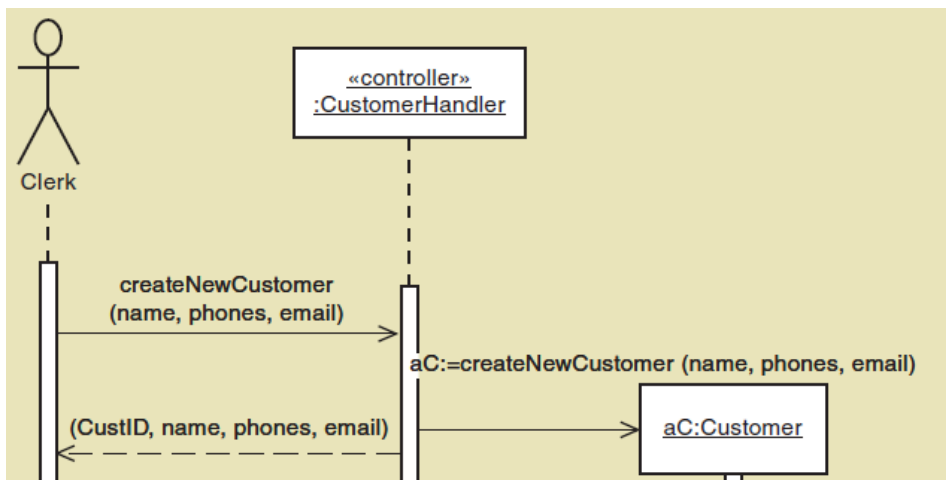
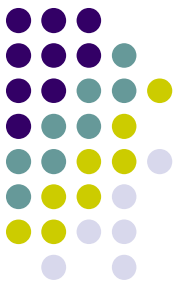
These domain classes handle data access, so this is a two layer architecture





**Please write the body of
CustomerHandler class in Java.**

正常使用主观题需 2.0 以上版本雨课堂



Public Class CustomerHandler {

Private Customer aC;

Public cusInfo createNewCustomer(string name, string phone, string email) {

 aC = Customer.createNewCustomer(name,phone,email);

 return aC.getInfo();

}

Public String enterAddress(string address) {

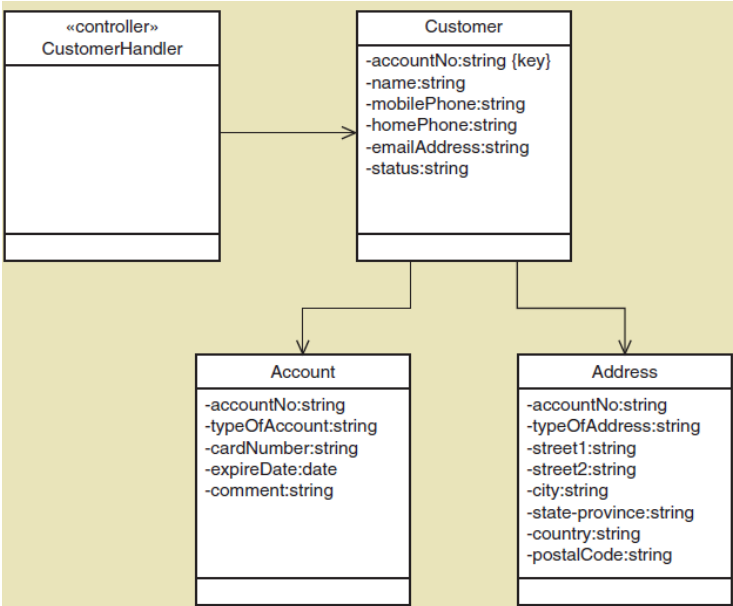
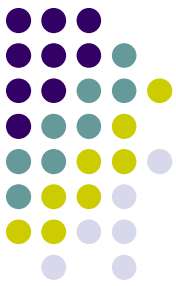
String s = aC.enterAddress(address);

Return s;

}

...

}



```
Public class Customer {
```

```
    Public class cusInfo{
```

```
        Public int CustID;
```

```
        Public string name;
```

```
        Public string phone;
```

```
        Public string email;
```

```
    }
```

```
    Private CustTable table = new CustTable();
```

```
    Private cusInfo = new cusInfo();
```

```
    Public static Customer createNewCustomer(string name, string phone,
    string email) {
```

```
        Customer Obj=new Customer();
```

```
        Obj.setInfo(string name, string phone, string email);
```

```
        Return obj;
```

```
    }
```

```
    Public void setInfo(string name, string phone, string email) {
```

```
        cusInfo.name = name;
```

```
        cusInfo.phone = phone;
```

```
        cusInfo.email = email;
```

```
        cusInfo.CustID = Table.insert(this);
```

```
    }
```

```
    Public cusInfo getInfo(){
```

```
        Return cusInfo;
```

```
    }
```

```
    Public string enterAddress(string addr)
```

```
    {
```

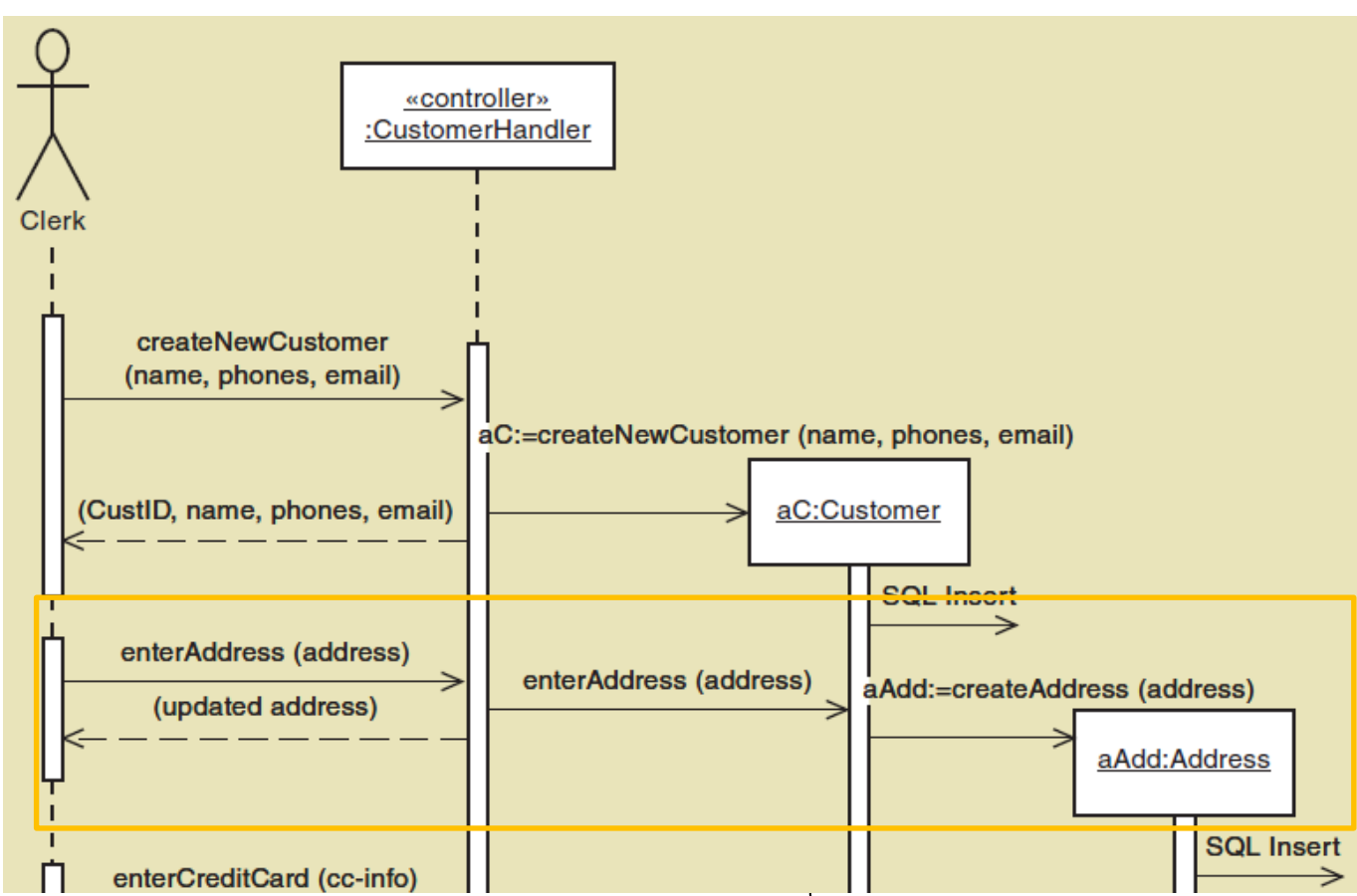
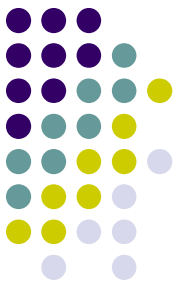
```
        Address aAdd = Address.createAddress(addr);
```

```
        Return aAdd.insert(this);
```

```
    }
```

```
    ...
```

```
}
```



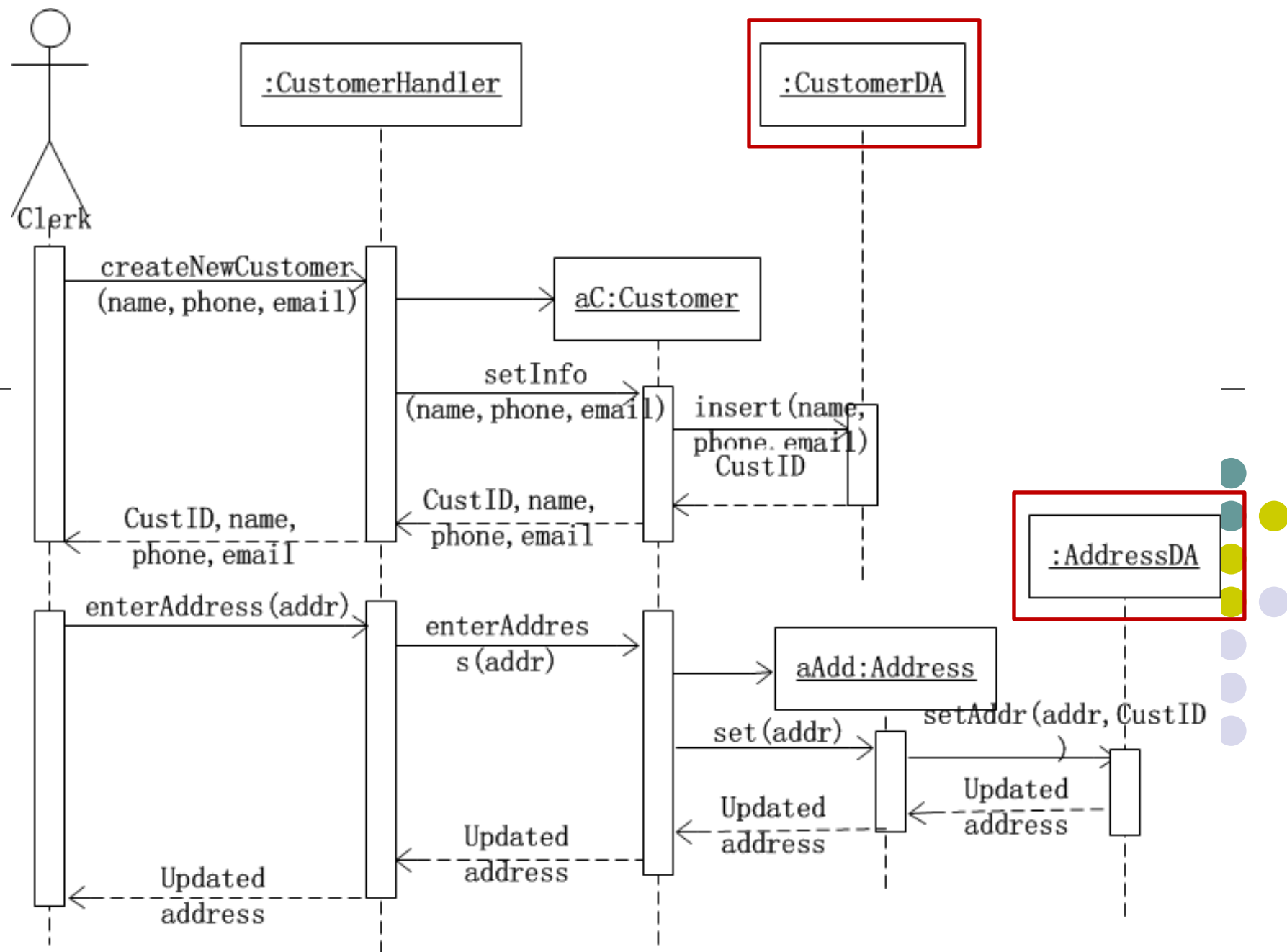
e());

customer(string name, string phone,

; phone, string email);

phone, string email) {

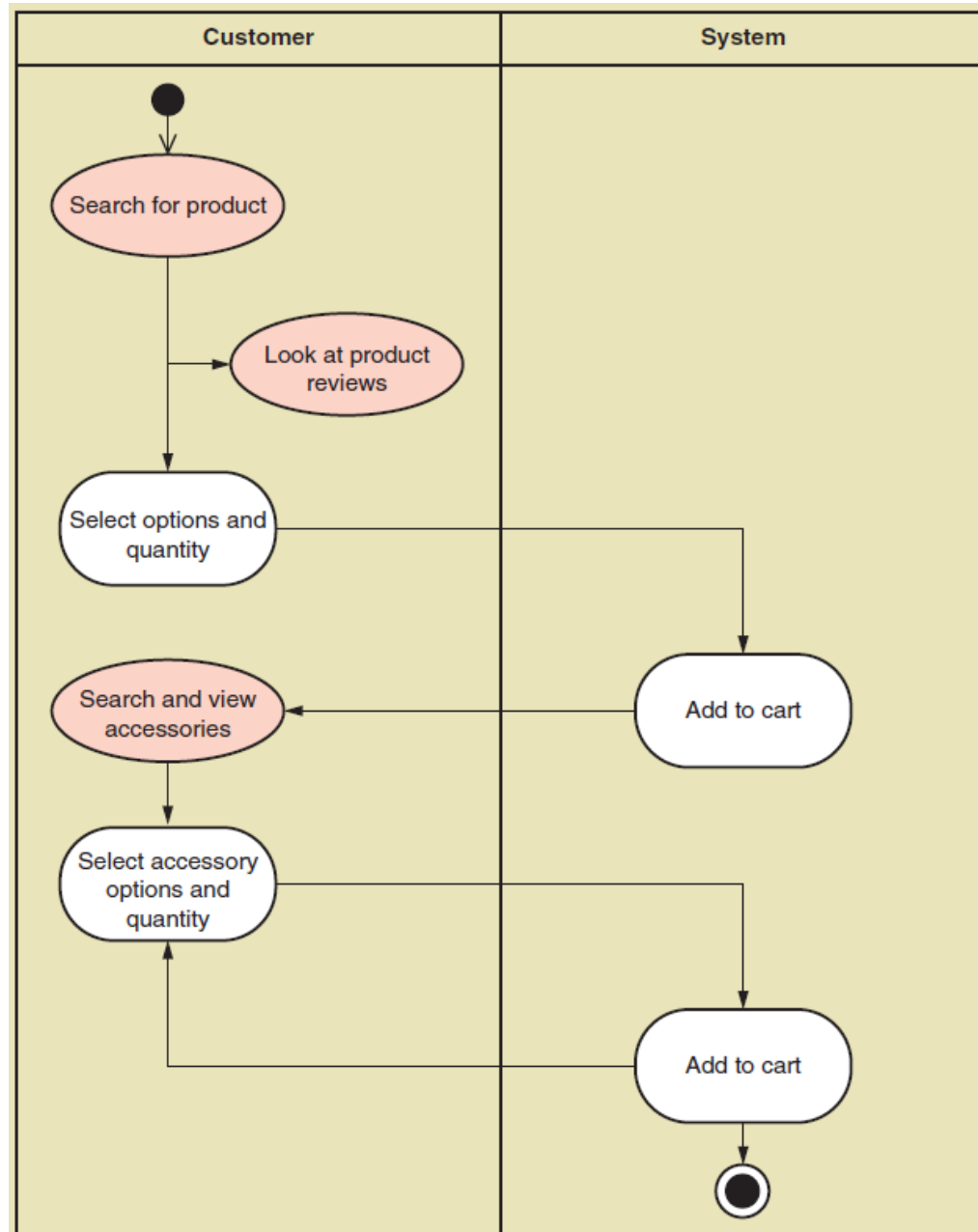
```
}
Public cusInfo getInfo(){
    Return cusInfo;
}
Public string enterAddress(string addr)
{
    Address aAdd = Address.createAddress(addr);
    Return aAdd.insert(this);
}
...
```



Fill Shopping Cart Use Case

Start with domain layer based on **activity diagram** and then **SSD**

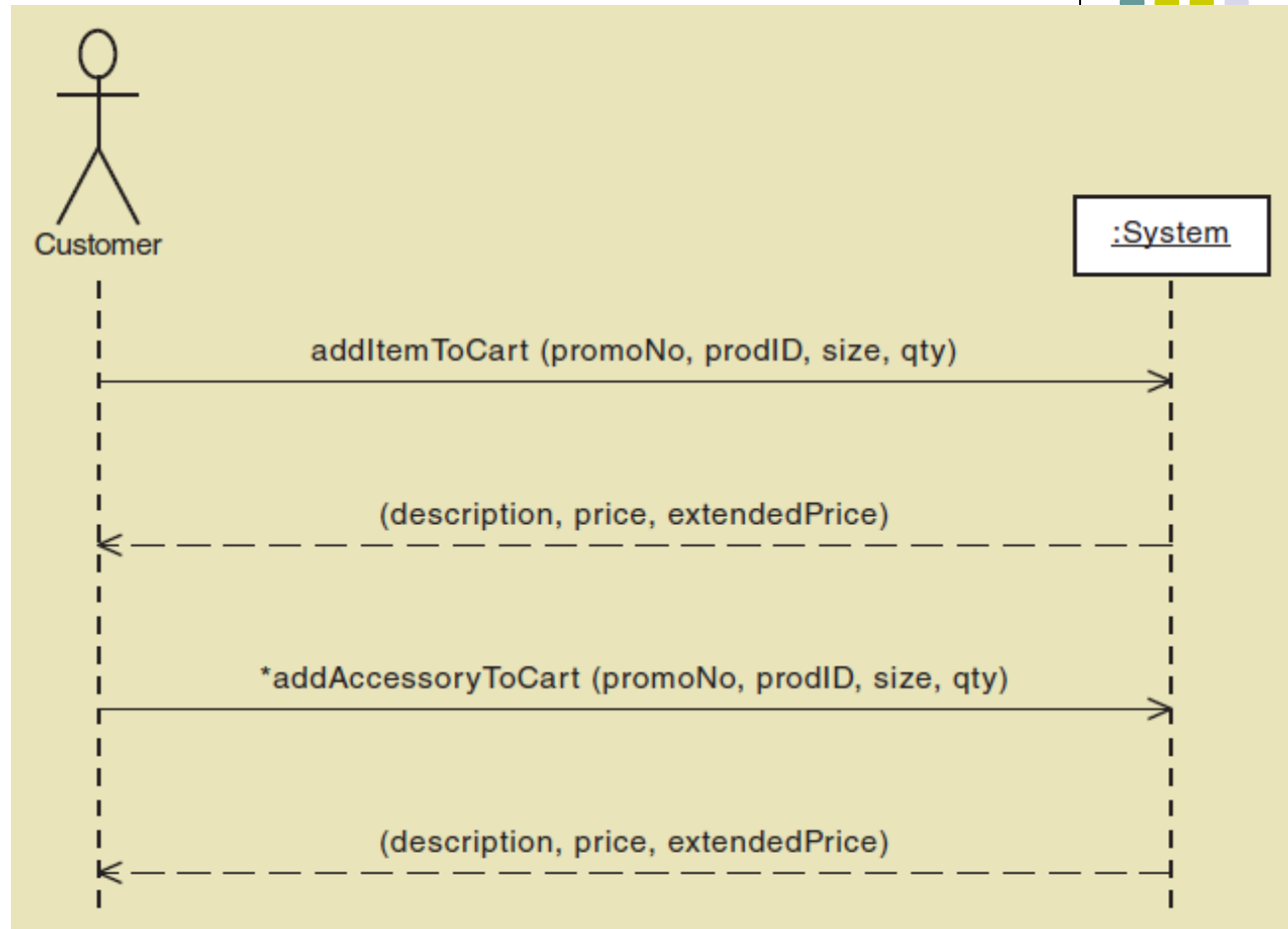
(从基于活动图的域层开始，然后是系统顺序图)





Fill Shopping Cart Use Case

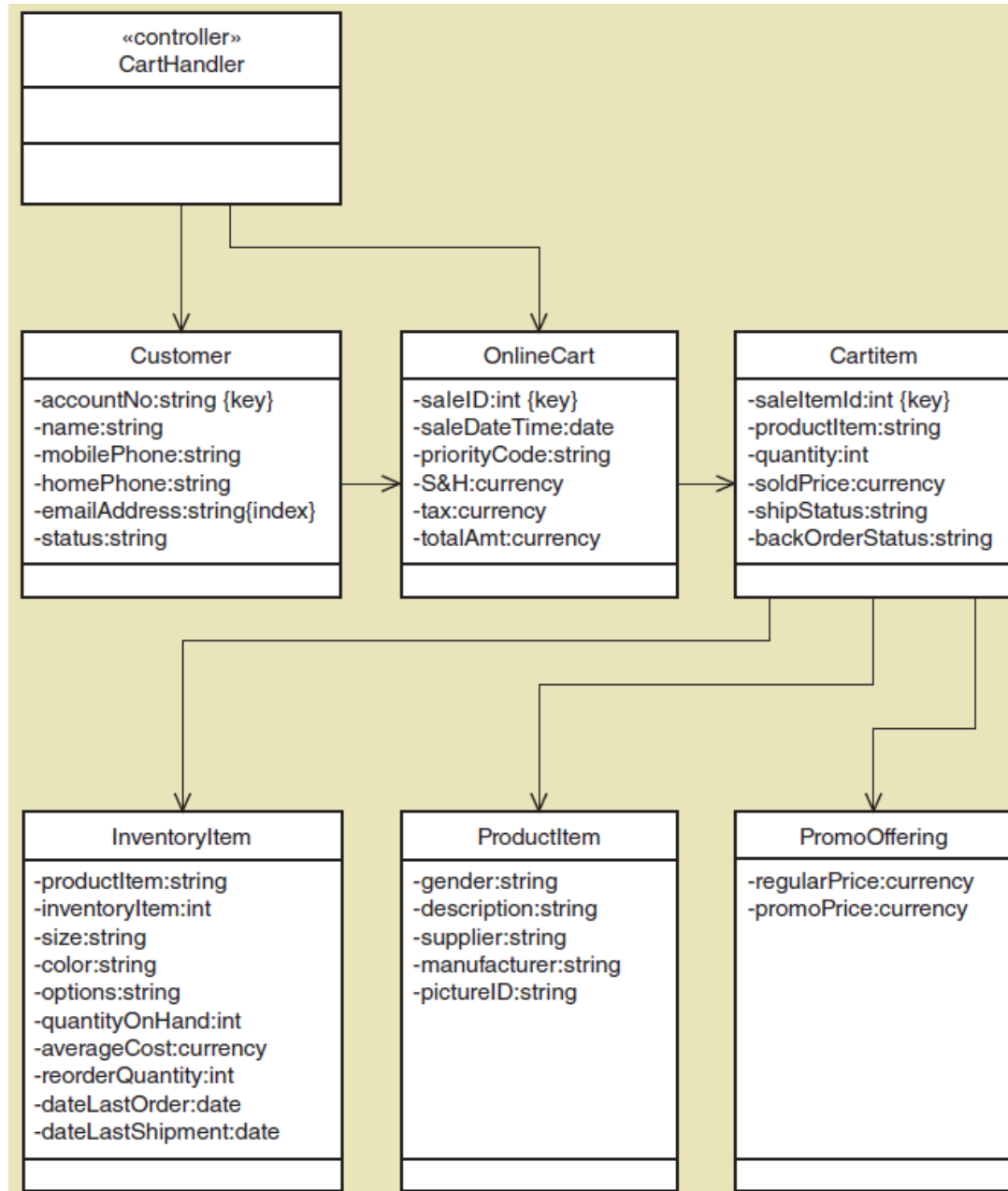
Start with domain layer based on activity diagram and then **SSD**



Fill Shopping Cart Use Case

First cut design class diagram

Select classes, elaborate attributes, add navigation visibility

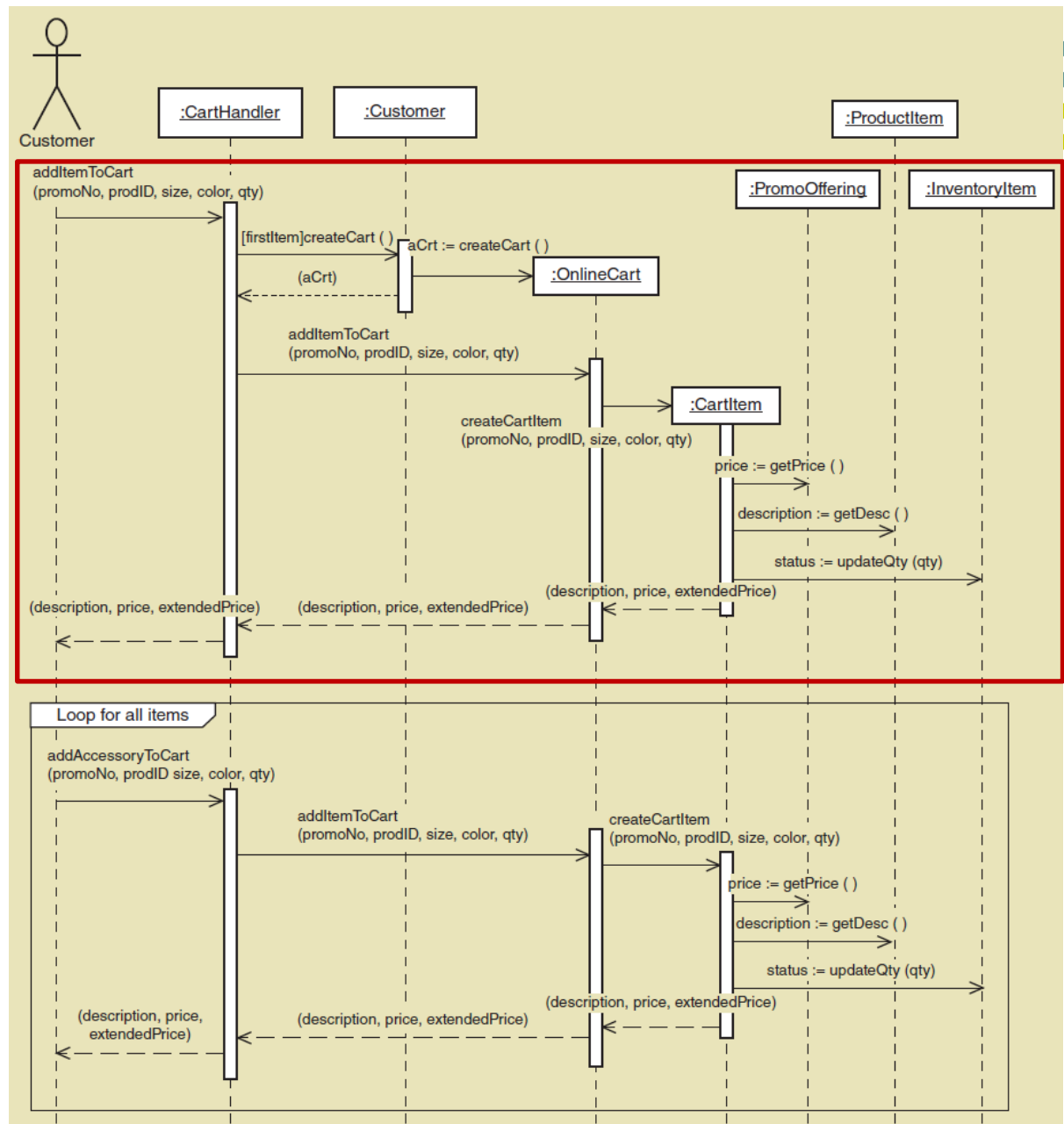


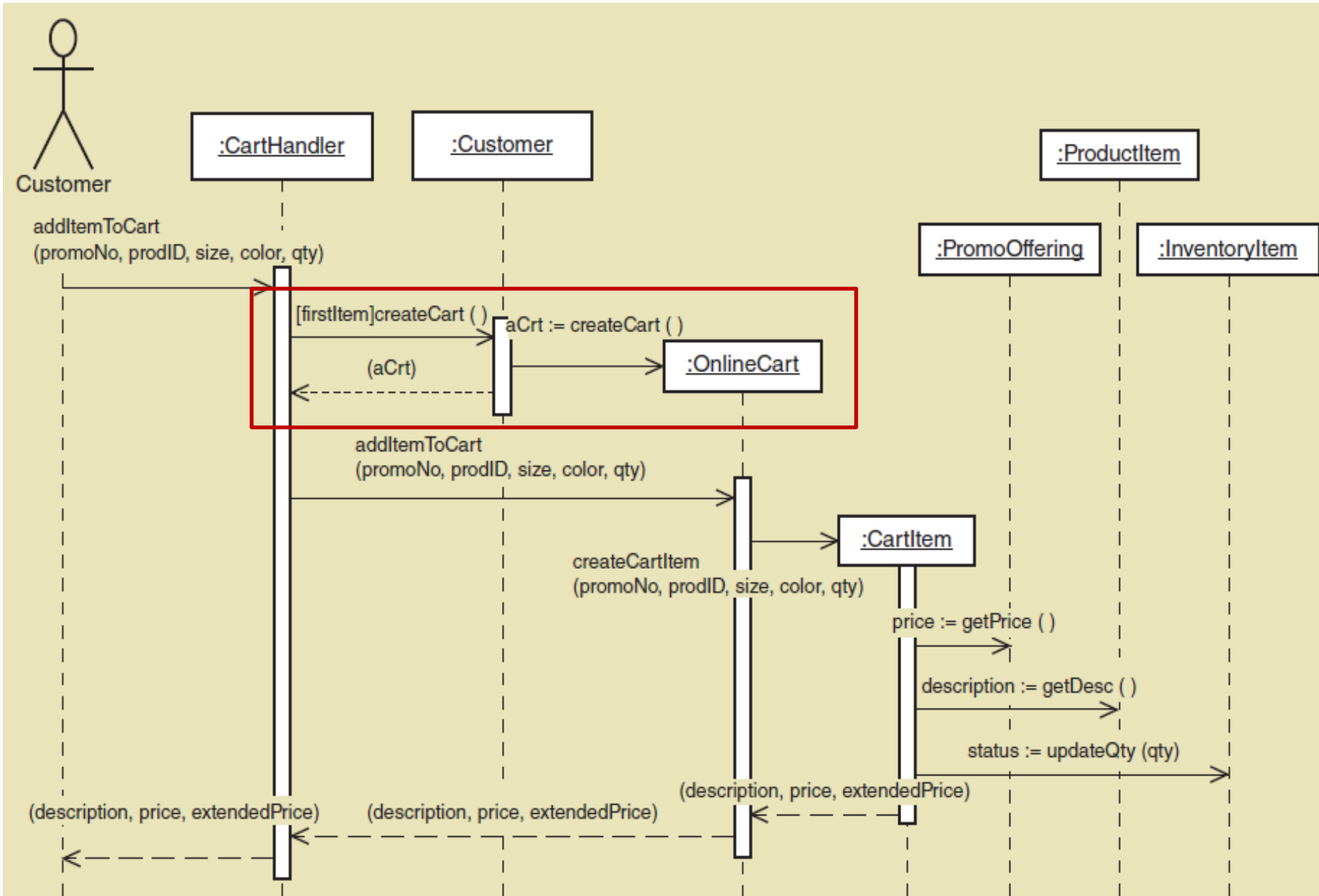
Fill Shopping Cart Use Case

First cut sequence diagram for domain layer. Will be three layer architecture.

:CarHandler is controller, other domain classes are included.

Note loop frame for adding each item to cart. The SSD used * notation (循环)





Assumptions

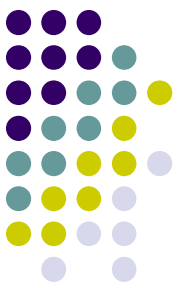


- Perfect technology assumption (完美技术假设) — First encountered for use cases. We don't include messages such as the user having to log on. (我们没有包含诸如用户登录之类的信息)
- Perfect memory assumption (完美内存假设) — We have assumed that the necessary objects were in memory and available for the use case. In multilayer design to follow, we do include the steps necessary to create objects in memory. (我们已经假设必要的对象在内存中并且对用例可用)

Assumptions



- Perfect solution assumption (完美解假定) — The first-cut sequence diagram assumes no exception conditions.
(初步顺序图假定不存在异常, 其中没有处理查询的产品为空的逻辑)
- Separation of responsibilities (职责分离) — Design principle that recommends segregating classes into separate components based on the primary focus, such as user interface, domain, and data access (建议根据主要关注点 (如用户界面、问题域和数据访问) 将类分类为独立组件的设计原则)

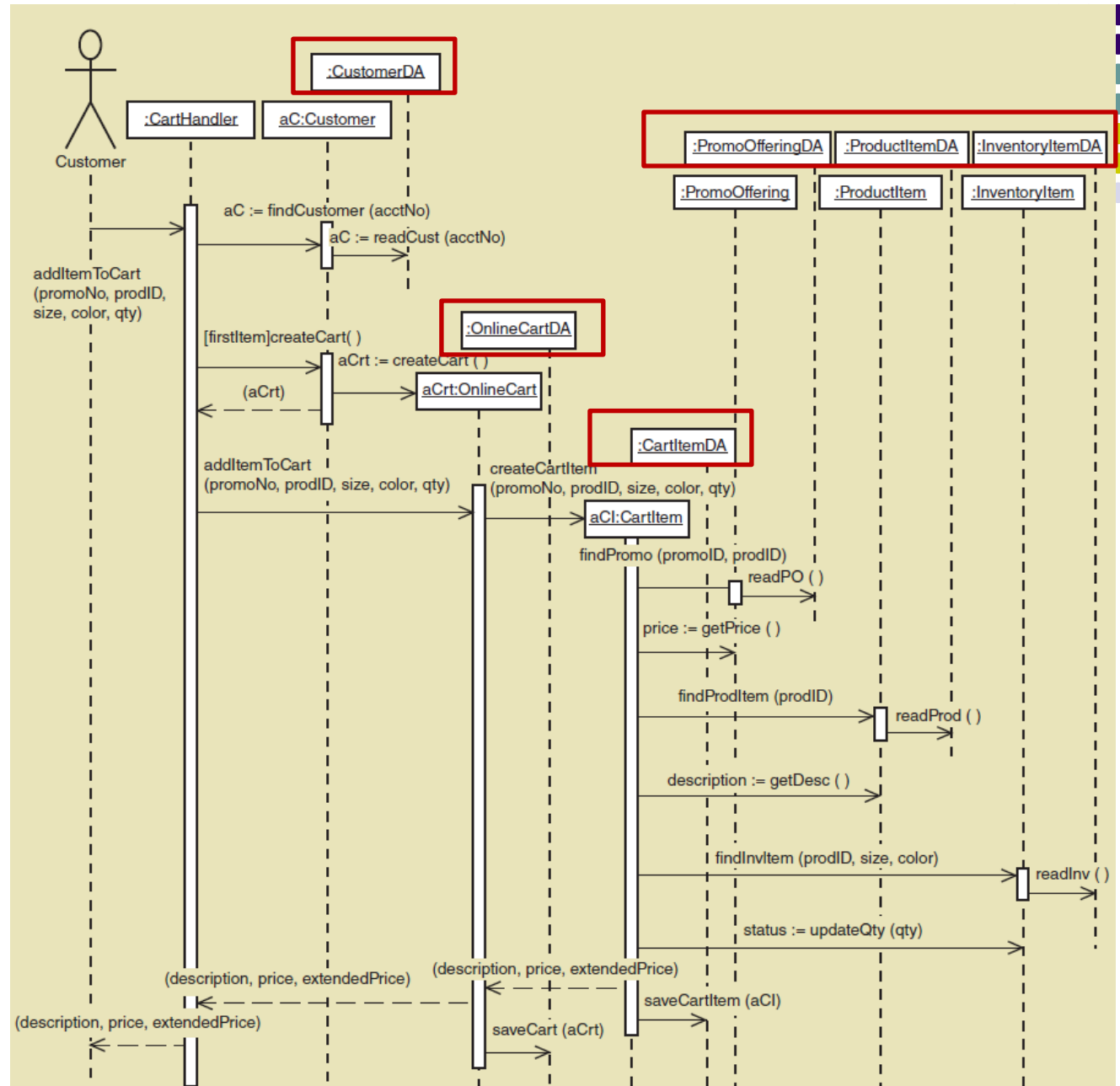


Developing a Multilayer Design

- View Layer (视图层)
 - With users, there is always a view layer. Add forms or pages
- Data Access Layer (数据访问层)
 - Persistent classes need some mechanism for storing and retrieving their state from a database (持久类需要某种机制来从数据库中存储和检索它们的状态)
 - Separate layer required when the business logic is complex and there is a need to separate connection to database and SQL from the domain classes. (当业务逻辑很复杂，并需要从域类中分离到数据库和 SQL 的连接时，需要单独的层)
 - *Create customer account* use case might not need a separate layer as domain classes can handle data (创建客户账户用例可能不需要单独的层，因为域类可以处理数据)
 - For each domain class, add a data access class to get data and save data about an instance (对每个域类添加数据访问类)

Adding Data Access Layer

Fill shopping Cart use case





Data Access Notes

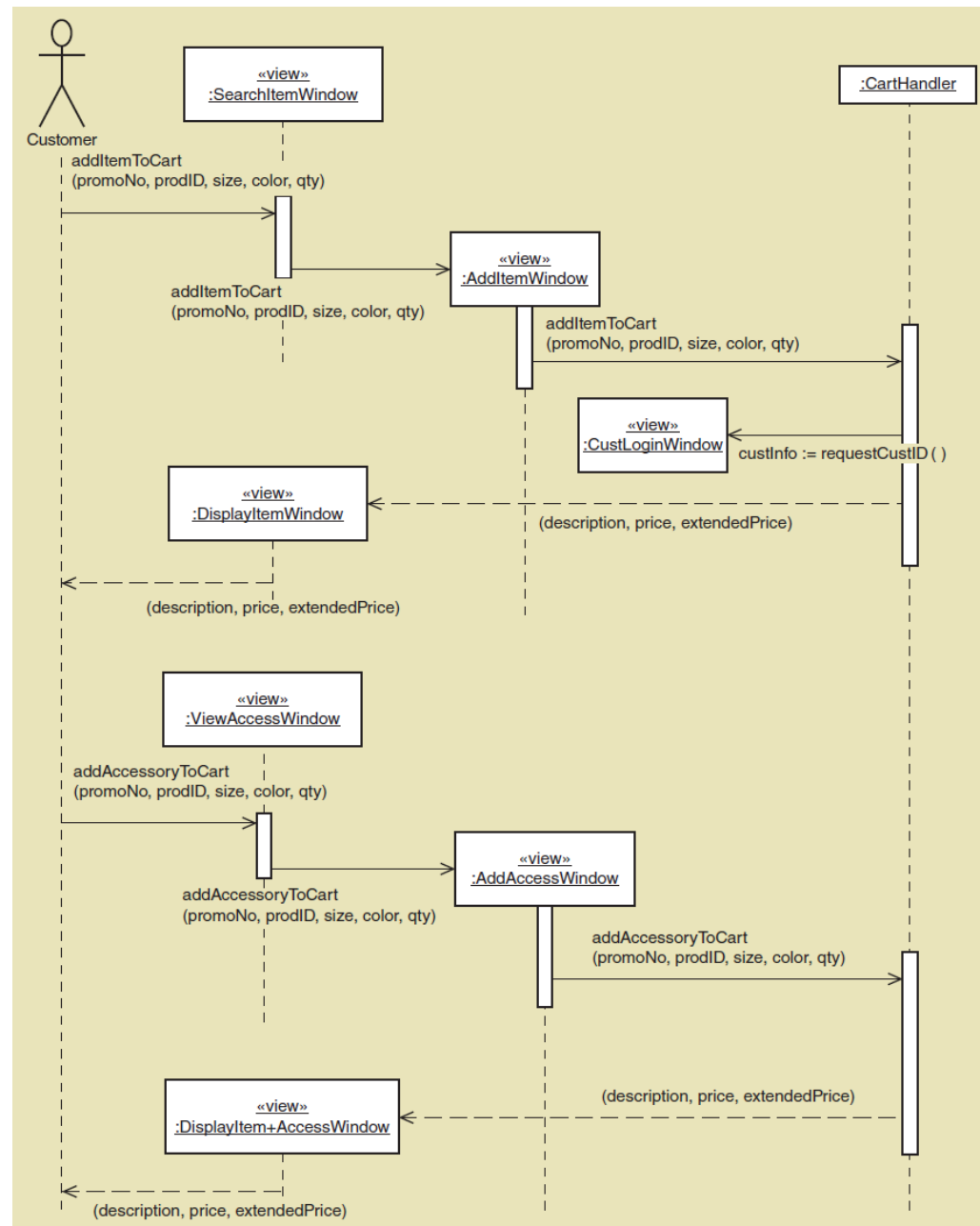
- :CartHandler findCustomer(acctN0) message to Customer class means Customer should create a new instance named aC, send a message to :CustomerDA asking it to **read info from the database** for a customer with that account number, and then populate the new customer instance with the attribute values from the database.
- :CartHandler findCustomer 发消息给 Customer 类意味着 Customer 应该创建一个名为 aC 的新实例
- 向 :CustomerDA 发送消息，要求它从数据库中读取具有该账号的客户的信息，然后用数据库中的属性值填充新的客户实例。

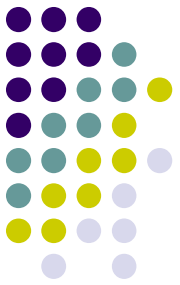
Adding View Layer

Fill shopping Cart use case

Sequence diagram just shows View interaction with the controller

View classes don't need to even know about the domain or data access layer

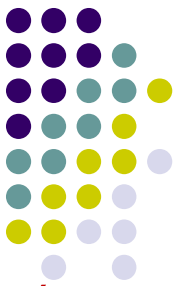




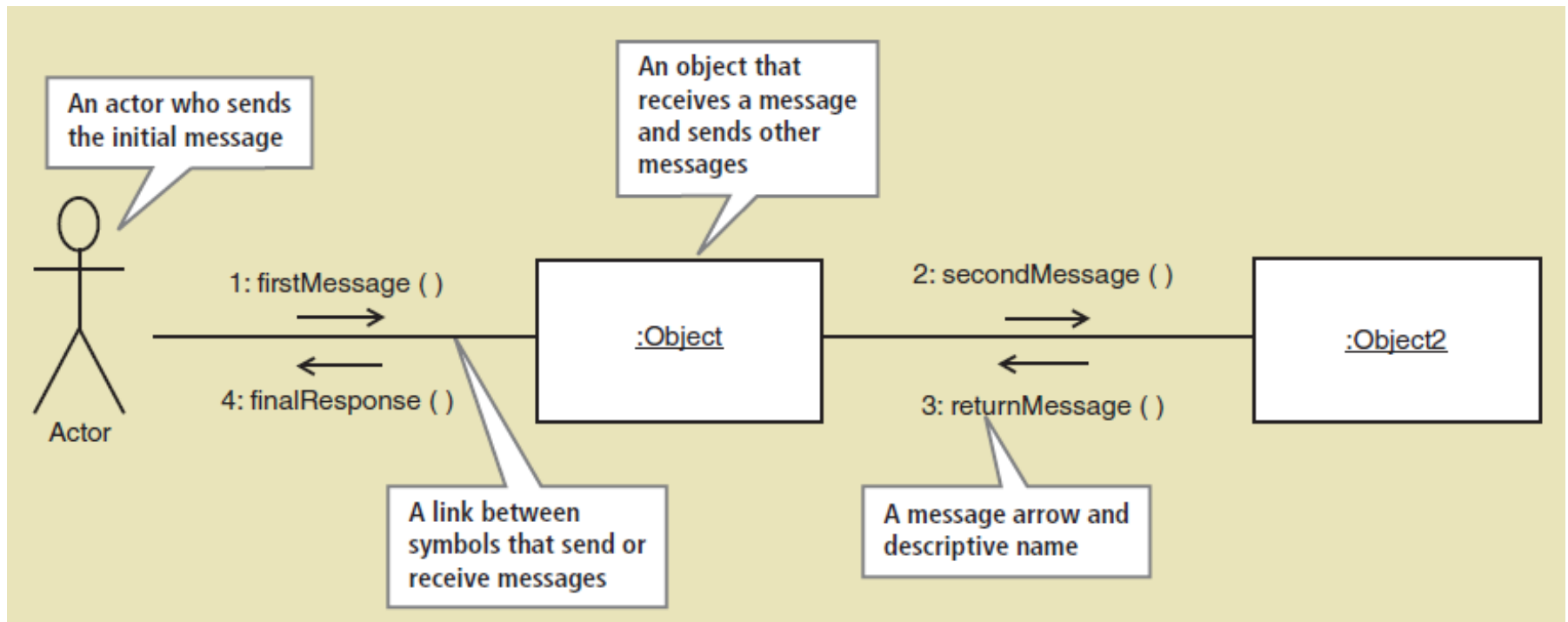
Chapter 11 Outline

- Detailed Design of Multilayer Systems
- Use Case Realization with Sequence Diagrams
- Designing with Communication Diagrams
(设计通信图 / 协作图)
- Updating and Packaging the Design Classes
- Design Patterns

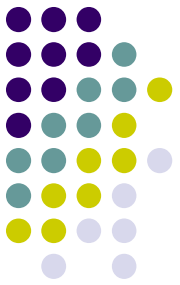
Designing with UML Communication Diagrams



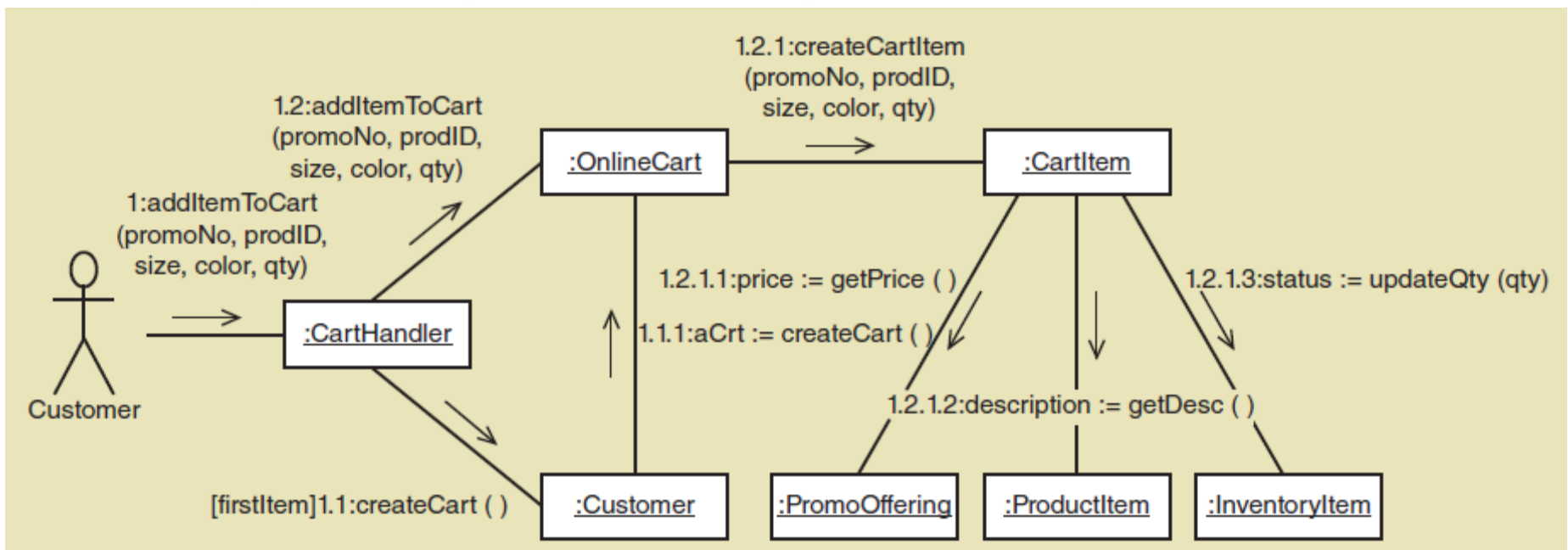
- Shows the same information as a sequence diagram (显示与顺序图相同的信息)
- Symbols used in a communication diagram:

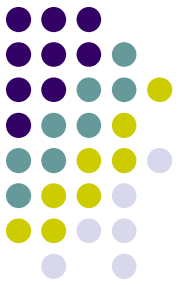


Communication Diagram for *Fill Shopping Cart* Use Case



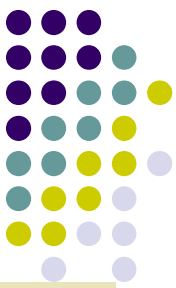
- This diagram should match the domain layer sequence diagram shown earlier
- [true/false condition]sequence-number: return-value := message-name(parameter-list)





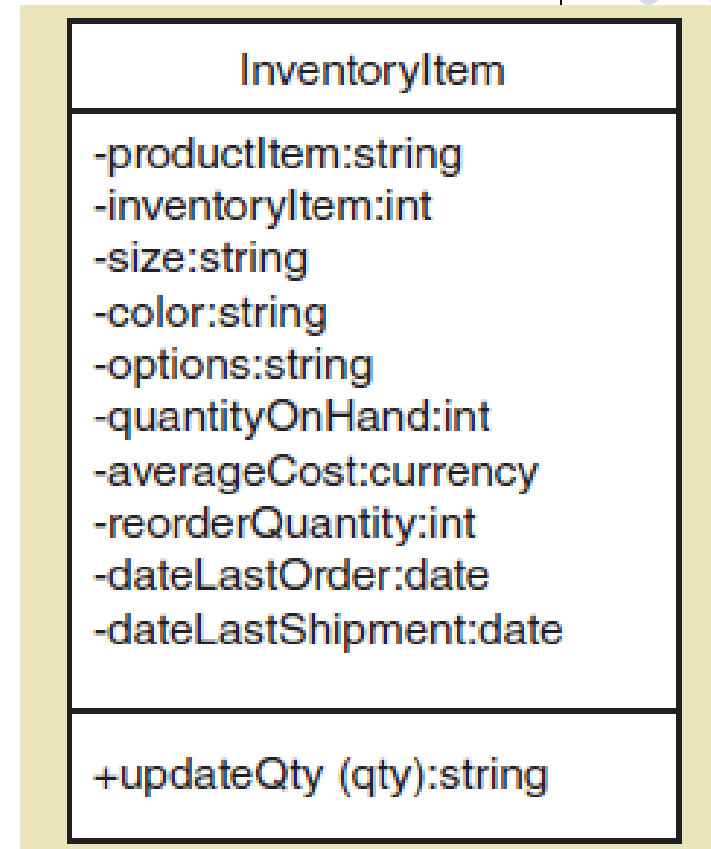
Chapter 11 Outline

- Detailed Design of Multilayer Systems
- Use Case Realization with Sequence Diagrams
- Designing with Communication Diagrams
- Updating and Packaging the Design Classes
(更新和打包设计类图)
- Design Patterns



Updating the Design Classes

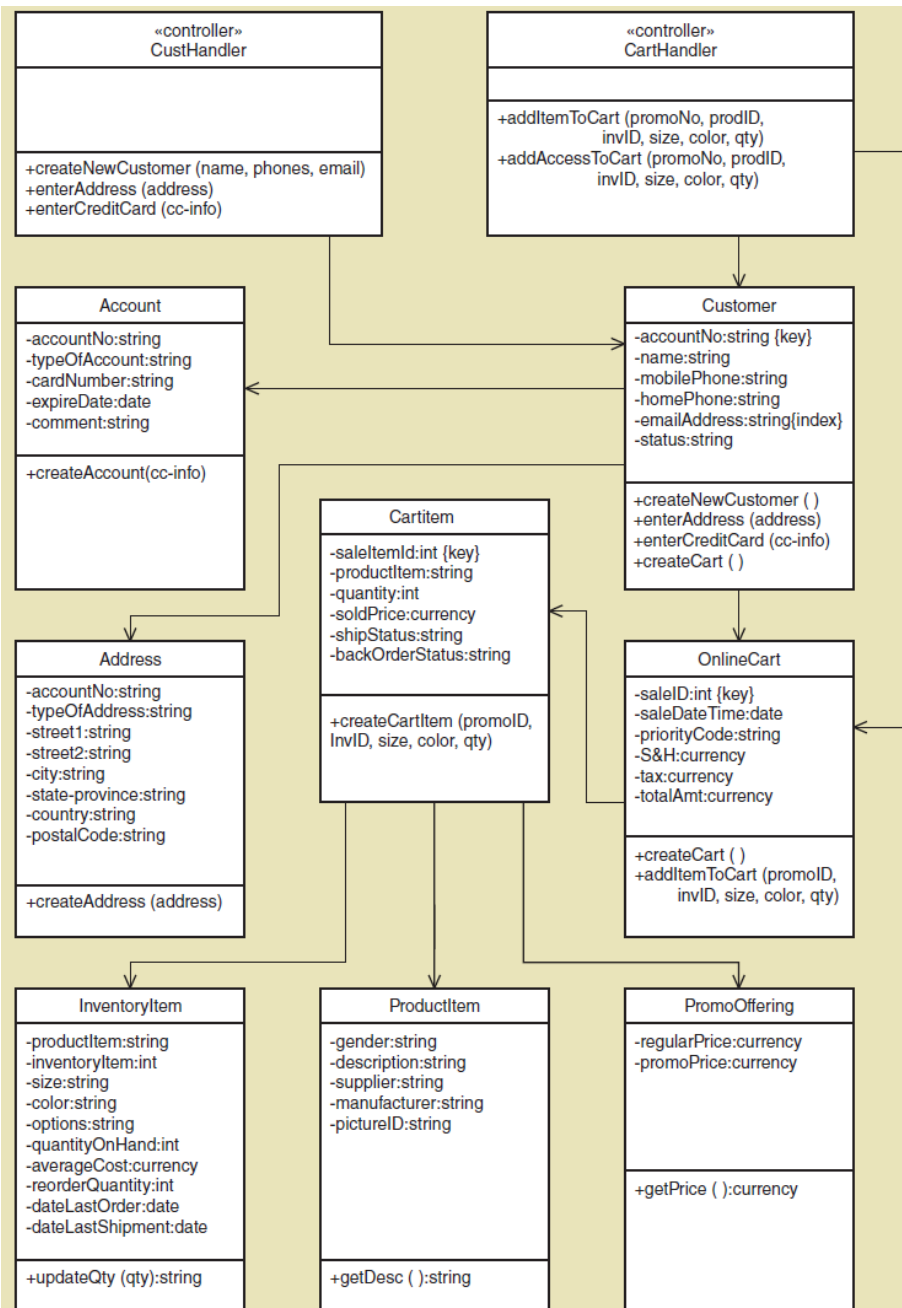
- Design class diagram (DCD) (设计类图) focuses on **domain layer** (域层)
- When an object of a class **receives** a message, that message **becomes a method** in the class on the DCD (当类的对象接收到消息时, 该消息将成为 DCD 上该类中的方法)



Updated Design Class Diagram for Domain Layer

After Create customer account use case and Fill shopping cart use case

完整的设计类图



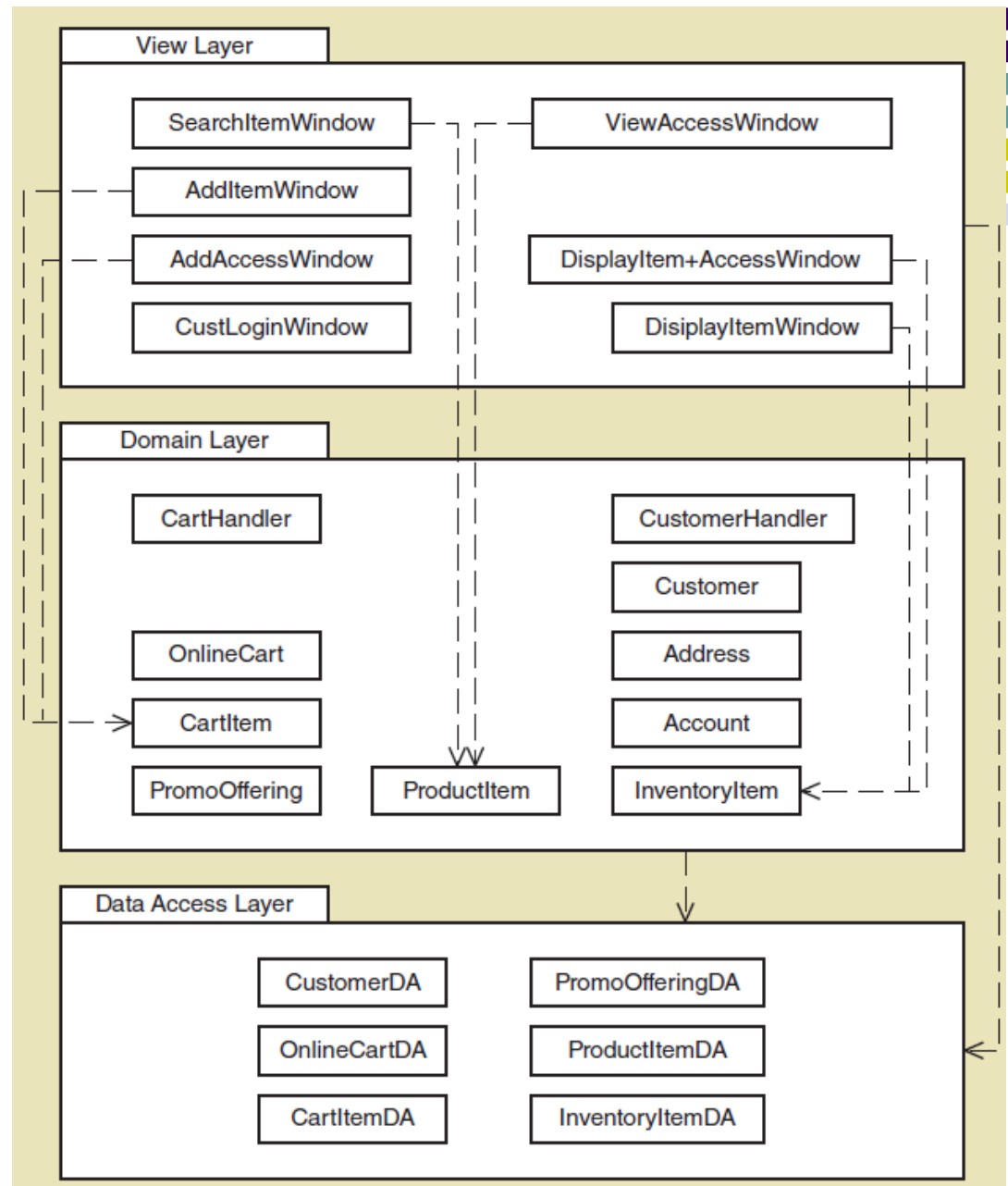
Structuring Components with UML Package Diagram

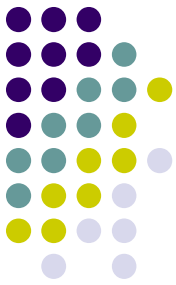
Three layer package diagram after two RMO use cases

Dashed line is dependency

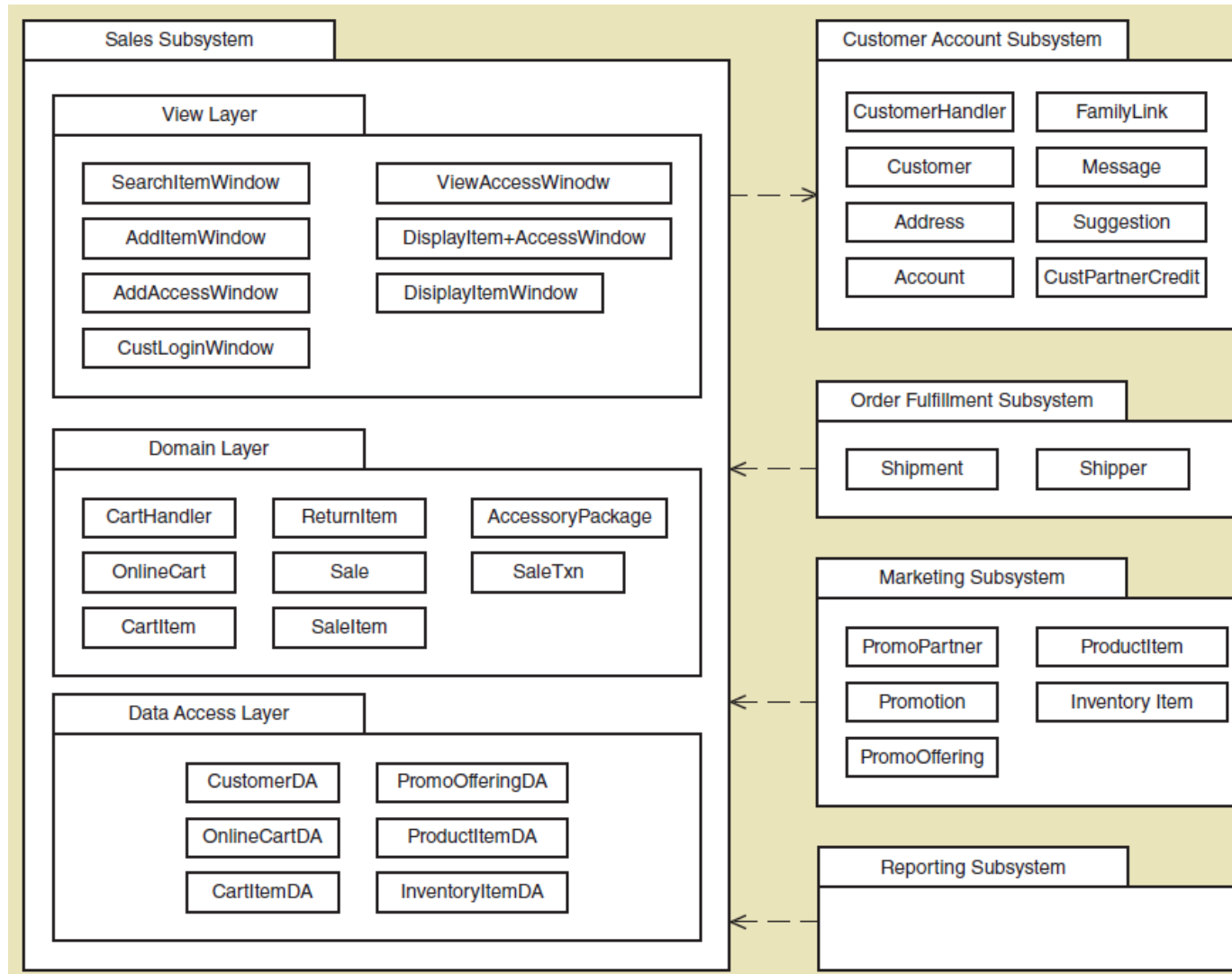
(包图使用的另一个符号是虚线箭头，它表示依赖关系)

Note: Java package or .NET namespace



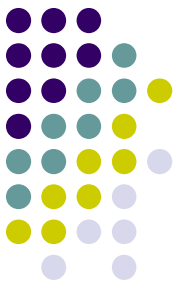


RMO Subsystem Package Diagram



Implementation Issues

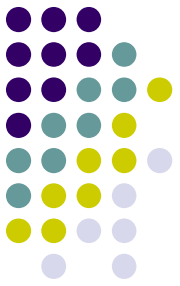
Three Layer Design



- **View Layer Class** Responsibilities (视图层完成以下任务)
 - Display electronic forms and reports. (展示结果)
 - Capture such input events as clicks, rollovers, and key entries. (捕捉输入)
 - Edit and validate input data. (编辑并校验)
 - Forward input data to the domain layer classes. (将数据传递给域层类)
 - Start and shut down the system. (启动与关闭系统)

Implementation Issues

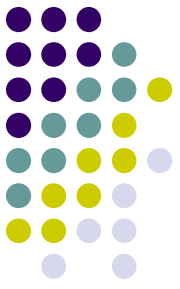
Three Layer Design



- Domain Layer Class Responsibilities (域层任务：)
 - Create problem domain (persistent) classes. (创建问题域类 (持久类))
 - Process all business rules with appropriate logic. (以适当的逻辑处理所有的业务规则)
 - Prepare persistent classes for storage to the database. (准备持久类以便数据库存储)

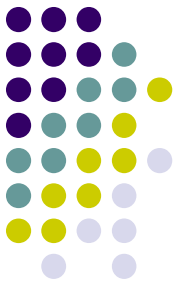
Implementation Issues

Three Layer Design



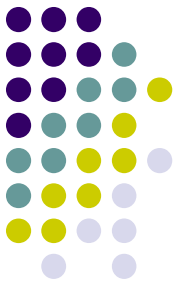
- Data Access Layer Class Responsibilities (数据访问层)
 - Establish and maintain connections to the database. (建立并维护数据库之间的连接)
 - Contain all SQL statements. (包含所有的 SQL 语句)
 - Process result sets (the results of SQL executions) into appropriate domain objects. (处理结果集 (SQL 语句的执行结果), 并赋给合适的域对象)
 - Disconnect gracefully from the database. (实施断开与数据库的连接)

Summary



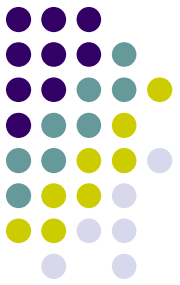
- This chapter went into more detail about use case realization and three layer design to extend the design techniques from last chapter （本章更详细地介绍了用例实现和三层设计，以扩展上一章的设计技术）
- Three layer design is an architectural design pattern, part of the movement toward the use of design principles and patterns. （三层设计是一种架构设计模式，是设计原则和模式使用运动的一部分。）
- Use case realization is the design of a use case, done with a design class diagram and sequence diagrams. Using sequence diagrams allows greater depth and precision than using CRC cards. （用例实现是用例的设计，通过设计类图和顺序图来完成。使用顺序图比使用 CRC 卡具有更大的深度和精度。）

Summary (continued)



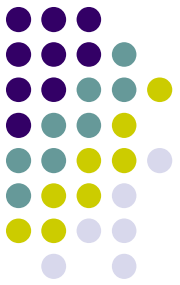
- Starting with the business logic/domain layer, domain classes are selected and an initial design class diagram is drawn.
(从业务逻辑 / 领域层开始, 选择领域类并绘制初始设计类图。)
- The systems sequence diagram (SSD) from analysis is expanded by adding a use case controller and then the domain classes for the use case. (系统顺序图 (SSD) 通过添加用例控制器和用例的域类来扩展。)
- Messages and returns are added to the sequence diagram as responsibilities are assigned to each class. (消息和返回被添加到顺序图中)
- The design class diagram is then updated by adding methods to the classes based on messages they receive and by updating navigation visibility. (通过根据接收到的消息向类添加方法和更新导航可见性来更新设计类图)

Summary (continued)



- Simple use case might be left with two layers if the domain classes are responsible for database access. More complex systems add a data access layer as a third layer to handle database access (如果域类负责数据库访问, 那么简单的用例可能只剩下两层。更复杂的系统添加数据访问层作为第三层来处理数据库访问)
- The view layer can also be added to the sequence diagram to show how multiple pages or forms interact with the use case controller. (视图层还可以添加到顺序图中, 以显示多个页面或表单如何与用例控制器交互)
- The UML communication diagram is also used to design use case realization and it shows the same information as a sequence diagram. (协作图也用于设计用例实现, 它显示了与顺序图相同的信息。)

Summary (continued)



- The UML package diagram is used to structure the classes into packages, usually one package per layer. The package diagram can also be used to package layers into subsystems. (UML 包图用于将类构造成包，通常每层一个包。包图还可以用于将层打包到子系统中。)
- Design patterns are a standard solutions or templates that have proven to be effective approaches to handling design problems. The design patterns in this chapter include **controller, adapter, factory, and singleton** (设计模式是一种标准解决方案或模板，已被证明是处理设计问题的有效方法。本章的设计模式包括控制器、适配器、工厂和单例模式)